Combinatorics II

Kon Yi

May 3, 2026

Abstract

Lecture note of Combinatorics II.

Contents

Chapter 1

Sorting Algorithm

Lecture 1

24 Feb.

Problem 1.0.1. Given n distinct numbers $a_1, a_2, \ldots, a_n$, sort them in increasing order.

- Input: n distinct numbers $a_1, a_2, \ldots, a_n$.
- Output: a permutation $\pi \in S_n$ s.t.

$$a_{\pi(1)} < a_{\pi(2)} < \cdots < a_{\pi(n)}.$$

Observation. Given x, y, is $x < y$?

Remark 1.0.1. When analyzing an algorithm, we consider the worst-case input and count operations, where operations are defined according to context.

Algorithm 1.1: Brute force

1 Go through all $\pi \in S_n$ one by one and test each permutation to see if it is correct.

Remark 1.0.2. For running time, testing a permutation takes $\leq n-1$ comparisons, and there are $n!$ possible permutations, so

$$\#\text{ of comparisons} \leq n!(n-1).$$

Question. What is efficient? How does the running time scale as the input size n increases? Ideally, the running time should be polynomial $O(n^c)$ for some $c \in \mathbb{R}$.

Algorithm 1.2: Insertion Sort

1 Maintain a sorted partial list, add one element at a time until the entire list is sorted.

Algorithm 1.3: Binary insertion sort

1 `// As before, we maintain a sorted prefix.`
2 Insert a_i into sorted list $a_1 < a_2 < \cdots < a_{i-2} < a_{i-1}$. Compare a_i to the middle element. If similar, insert into the first half. If bigger, insert into the second half. `// Either way reduce number of possible positions by half each time`

Remark 1.0.3. Thus, we can insert in $\approx \log_2 i$ many comparisons each time, and thus in total we need $O(n \log_2 n)$ comparisons.

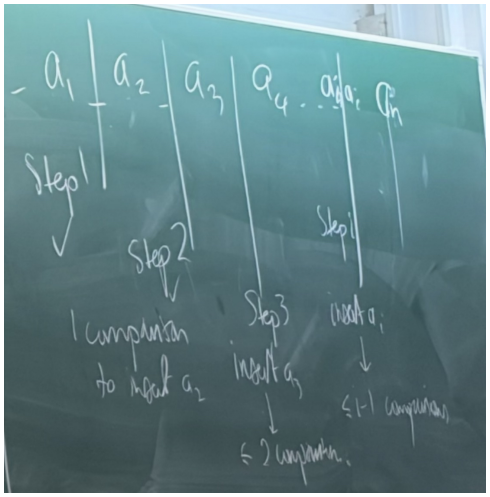
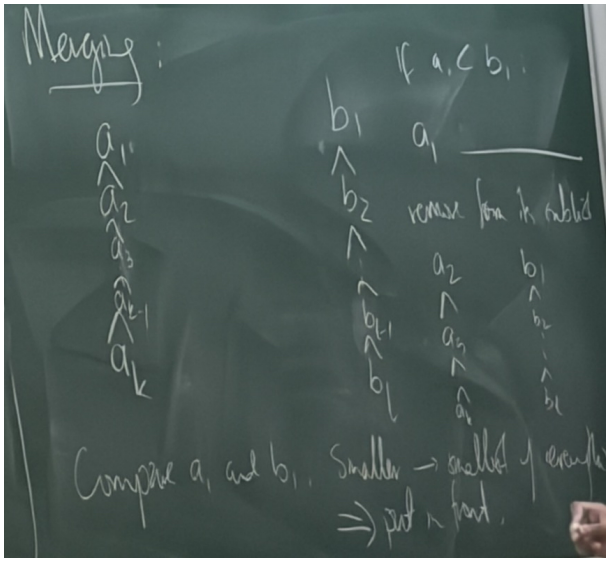


Figure 1.1: Insertion sort

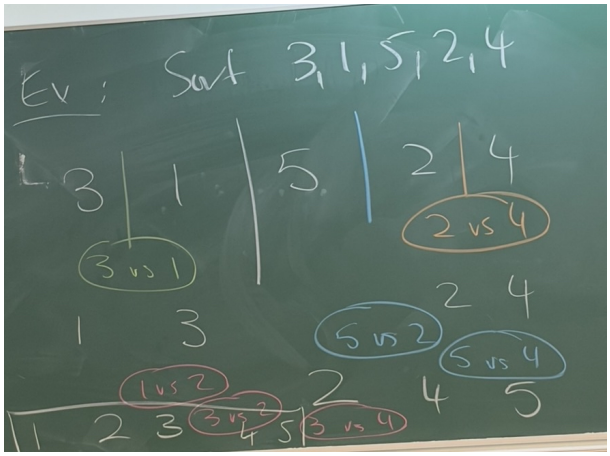
Algorithm 1.4: Merge Sort

1 `// Recursive algorithm`
2 Split the list into two equal halves.
3 Sort each half.
4 Merge the two sorted lists into one. `// Compare the head of two list, remove the smaller one from original list then put it in the back of a new list, which is empty initially, and repeat this comparison step.`

Remark 1.0.4. Number of comparison to merge $\leq n - 1$.



(a) Merging



(b) An example of merge sort

Remark 1.0.5. Running time: Let $T(n)$ be the worst case running time for merge sort on a list of

n numbers. Then $T(1) = 0$, and we know

$$T(n) \leq \underbrace{T\left(\lfloor \frac{n}{2} \rfloor\right)}_{\substack{\text{sorting the} \\ \text{first half}}} + \underbrace{T\left(\lceil \frac{n}{2} \rceil\right)}_{\substack{\text{sorting the} \\ \text{second half}}} + \underbrace{n-1}_{\text{merge}}.$$

Now suppose $n = 2^k$, $k \in \mathbb{N}$, then

$$\begin{cases} T(1) = 0 \\ T\left(2^k\right) = 2T\left(2^{k-1}\right) + 2^k - 1 \text{ for } k \geq 1. \end{cases}$$

Thus,

$$\begin{aligned} T\left(2^k\right) &= 2T\left(2^{k-1}\right) + 2^k - 1 \\ &= 2\left[2T\left(2^{k-2}\right) + 2^{k-1} - 1\right] + 2^k - 1 \\ &= \cdots = 2^i T\left(2^{k-i}\right) + i2^k - \left(2^i - 1\right), \end{aligned}$$

and if we let $i = k$, then

$$T\left(2^k\right) = 2^k T(1) + k2^k - 2^k + 1 = k2^k - 2^k + 1 = n\log_2 n - n + 1.$$

Thus, $T(n) \leq n\log_2 n - n + 1$ when $n = 2^k$.

Question (Lower bounds). Can we do better?

Answer. No, but to prove this, we need to show that any other algorithm, no matter how weird, takes as long. ⊛

Question. Suppose we have an algorithm A for sorting n numbers. Now can we lower-bound its runtime?

Information Theory approach

Each comparison gives ≤ 1 bit of information. At the end, we get the information of the permutation to sort the numbers, and since there are $n!$ possible permutations, so the number of time needed is $\geq \log_2 n!$.

Lecture 2

3 March.

An arbitrary sorting algorithm is a function that takes the sequence of answers to our queries (is $x_i < x_j$?) and returns a permutation $\pi \in S_n$ s.t. $x_{\pi(1)} < x_{\pi(2)} < \cdots < x_{\pi(n)}$. Suppose the algorithm has a worst-case running time of k. Then this function has at most 2^k different outputs since such binary decision tree can has at most 2^k leaves and each leaf represent a possible permutation.

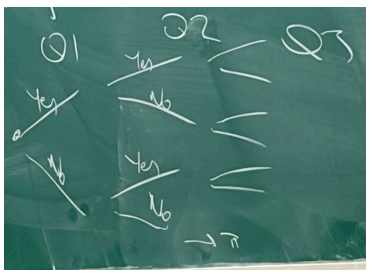


Figure 1.3: The decision tree

The entirety of S_n has to be in the image, so $2^k \geq |S_n| = n!$. Hence, $k \geq \log_2(n!) = (1 + o(1))n\log_2 n$.

Alternative arguments (playing against the devil)

You ask the comparisons to the devil, whose goal is to make the game go on as long as possible, The devil maintains a list of all possible permutations. Every time we do a comparison, we split the list into two: Those whose the answer is "yes" and those where it is "no". The devil chooses the answer with the bigger part. Game ends when the list has size 1. Let L_i be the list of possible permutations after i comparisons. $L_0 = S_n$, and for all $i \geq 1$,

$$|L_i| \geq \frac{1}{2}|L_{i-1}|.$$

Suppose one algorithm requires k comparisons in the worst case, then

$$1 \geq |L_k| \geq \frac{1}{2}|L_{k-1}| \geq \frac{1}{4}|L_{k-2}| \geq \cdots \geq \frac{1}{2^k}|L_0| = \frac{n!}{2^k},$$

so $2^k \geq n!$, which shows $k \geq (1 + o(1))n \log_2 n$.

Chapter 2

Connectivity and Spanning Tree

Definition 2.0.1 (graph). A (simple) graph is an ordered pair of sets (V, E), where V is a set of vertices and $E \subseteq \binom{V}{2}$ is a set of edges, where an edge is an unordered pair of disjoint vertices.

Definition 2.0.2. A walk in a graph is a sequence

$$v_0 e_1 v_1 e_2 v_2 e_3 v_3 \dots e_n v_n$$

where $v_i \in V$ and $e_i = \{v_{i-1}, v_i\} \in E$ for all i. A trail is a walk that does not repeat any edges. A path is a trail/walk without repeated vertices. A circuit is a trail that starts and ends at the same vertex. A cycle is a circuit that doesn't repeat any internal vertex.

Definition 2.0.3. A graph $G = (V, E)$ is connected if for every $u, v \in V$, there is a path(walk) from u to v.

Definition 2.0.4. Given a graph $G = (V, E)$ and a vertex $v \in V$, the connected componnent of v is the maximal connected subgraph of G that contains v.

Remark 2.0.1. $G' = (V', E')$ is a subgraph of $G = (V, E)$ if $V' \subseteq V$ and $E' \subseteq E \cap \binom{V'}{2}$. It is induced if $E' = E \cap \binom{V'}{2}$. It is spanning if $V' = V$.

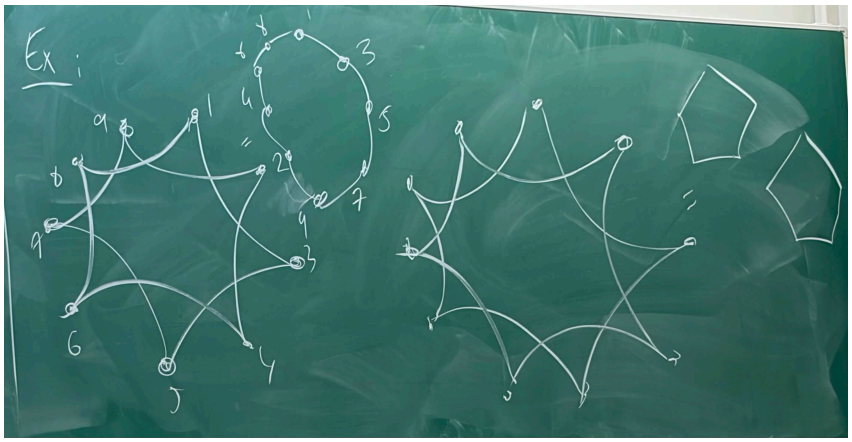


Figure 2.1: An example of connectivity (left one is connected but right one not)

Question. How can we identify the connected components of a graph?

Algorithm 2.1: Find the connected component containing v

Input: A graph $G = (V, E)$ and a vertex $v \in V$
Output: Connected component $C \subseteq V$ s.t. $v \in C$

```
// We will maintain two sets of vertices. Let W be the component so far, i.e.
 vertices we can reach from v whose neighborhood we have explored.
```
Initially $W_0 = \varnothing$. $Q_0 = \{v\}$. `// Q =queue of vertices to explore`
for *the i-th step* **do**
  if $Q_{i-1} = \varnothing$ **then**
    DONE. Return W as the component as v.
  else
    Let w be an arbitrary vertex in Q_{i-1}. $Q_i = Q_{i-1} \setminus \{w\}$, $W_i = W_{i-1} \cup \{w\}$.
    Explore the neighborhood of w: If u is a neighborhood of w and $u \notin Q \cup W_i$, then add it to Q_i.

Claim 2.0.1. The algorithm is finite.

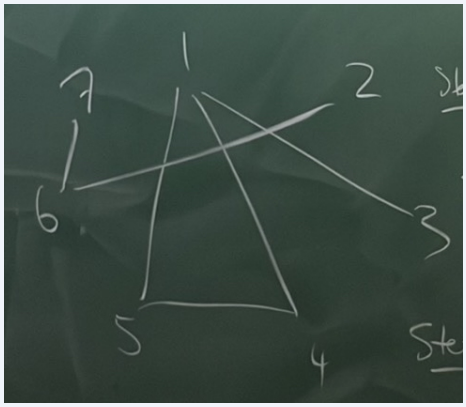
Proof. In every step, if we are not done, we add a distinct item to W (since Q_i and W_i are disjoint) and never remove it. Thus, we do at most $|V(G)|$ steps. ⊛

Claim 2.0.2. The algorithm is correct.

Proof. Let C_v be the component of v and W be the output of the algorithm. We show that $C_v = W$. We first show that $W \subseteq C_v$. Prove by induction on number of steps that we only add something to W if it was in Q. Also, we only add something to Q if it was a neighbor of an earlier member of W. By induction, there is a path from this vertex to v. Now we show $C_v \subseteq W$. Suppose not, then let $x \in C_v \setminus W$. Then there exists vx-path in G, say $v = v_0 v_1 v_2 \dots v_t = x$. Let j be the smallest index s.t. $v_j \notin W$. Then v_{j-1} is in W. Thus, we have explored its neighborhoods, so it would be in W. ⊛

It would be nice to not only know the components of v, but also paths that get there from v. Thus, in the algorithm, when exploring the neighborhood of a vertex $w \in Q_{i-1}$, if we add a neighbor u to Q_{i-1}, we keep track of the edge $\{w, u\}$ that was used to reach u.

Example 2.0.1. Now if $v = 4$ and the graph is the following figure:



Then we know

- Step 0: $Q_0 = \{4\}$, $W_0 = \{\}$.
- Step 1: $w = 4$, $Q_1 = \{1(\{1, 4\}), 5(\{4, 5\})\}$, and $W_1 = \{4\}$.
- Step 2: $w = 1, Q_2 = \{5(\{4, 5\}), 3(\{1, 3\})\}$, and $W_2 = \{4, 1(\{1, 4\})\}$.

- Step 3: $w = 5, Q_3 = \{3(\{1,3\})\}$, and $W_3 = \{4, 1(\{1,4\}), 5(\{4,5\})\}$.
- Step 4: $w = 3, Q_4 = \{\}$, and $W_4 = \{4, 1(\{1,4\}), 5(\{4,5\}), 3(\{1,3\})\}$.

Definition 2.0.5. A tree is a minimally connected graph, i.e. it is connected, but removing any edge disconnect it.

Proposition 2.0.1. Let T be an n-vertex graph. The following is equivalent:

(1) T is a tree.

(2) T is maximally acyclic.

(3) T is connected and has $n-1$ edges.

(4) T is acyclic and has $n-1$ edges.

Remark 2.0.2. Acyclic means no cycle.

Lecture 3

6 March.

If we modify Algorithm 2.1 so that we take the oldest element in Q, then we call this algorithm **Breadth-first search**.

If we modify Algorithm 2.1 so that we take the newest element in Q, then we call this algorithm **Depth-first search**.

2.1 Minimum weight spanning tree

Theorem 2.1.1 (Cayley). The complete graph K_n has n^{n-2} spanning trees.

We want to find the most efficient/cheapest spanning tree.

Problem 2.1.1 (Mininimum spanning tree).

- Input: Complete graph K_n and a weight function $w : E(K_n) \to \mathbb{R}_{\geq 0}$
- Output: A spanning tree T of K_n minimizing $\sum_{e \in E(T)} w(e)$.

Example 2.1.1. If G is an n-vertex graph (not necessarily complete). Define

$$w(e) = \begin{cases} 0, & \text{if } e \in E(G); \\ 1, & \text{if } e \notin E(G). \end{cases}$$

Then G is connected if and only if there exists a tree of weight 0.

Algorithm 2.2: Kruskal's algorithm(Theoretical way)

```
// Maintain F, and let F = ∅ initially.
Order the edges by weight, say e_1, e_2, ..., e_m.
for each i s.t. 1 ≤ i ≤ m do
    if F + e_i contributes a cycle then
        do nothing.
    else
        add e_i to F.
```

Algorithm 2.3: Kruskal's algorithm(Implementation way)

```
// Maintain F, and let F = ∅ initially, and C_v is the component containing v
   for all v ∈ V(G), where C_v = {v} initially for all v ∈ V(G).
Order the edges by weight, say e_1, e_2, ..., e_m.
for each i s.t. 1 ≤ i ≤ m do
    Let e_i = (u, v). if C_u = C_v then
        do nothing.
    else
        add e_i to F.
        Merge C_u and C_v.
```

Claim 2.1.1. At the end of the Kruskal's algorithm, F is a minimum spanning tree.

Proof. We have to show F is a spanning tree and F is minimal.

We first show F is a spanning tree. It suffices to show F has no cycles and connected. Note that in this algorithm we make sure every adds of an edge into F would not contribute a cycle, so the final F must has no cycle. Now we show F is connected. If not, then we have connected components. Since G is connected, there will be edges between components. Thus the algorithm would have added the first edge between these components. Now we know F is connected and acyclic. If F does not cover all vertices, say not covering v, then the edge between v and F would be added in the algorithm, so F covers all n vertcies, i.e. F is a spanning tree.

Now let T_k be the tree from Kruskal's algorithm. Let $T_{\min}$ be a minimum spanning tree that maximizes $|E(T_k) \cap E(T_{\min})|$. If $|E(T_k) \cap E(T_{\min})| = n - 1$, then T_k is a minimum spanning tree. Otherwise, consider the first edge (i.e. e_v with minimal v) in $T_k \Delta T_{\min}$, say e_i.

Claim 2.1.2. $e_i \in T_k$.

Proof. If $e_i \in T_{\min}$, then in the i-th step of Kruskal algorithm, we would have added e_i to T_k. (For $e_1, e_2, \ldots, e_{i-1}$, each edge is either in both tree or not in any of both, so e_i can be added to T_k in the algorithm since it will not contribute to a cycle (since $T_{\min}$ has no cycle)). Thus, $e_i \in T_k$. ⊛

Consider $T_{\min} + e_i$, which creates a cycle C. Thus, C must contain an edge e_j with $j > i$ (because all edges e_j with $j < i$ are common to both trees, and $C \nsubseteq T_k$). Let $T' = T_{\min} + e_i - e_j$, then T' is a spanning tree with smaller weight sum, which is a contradiction. ⊛

Lecture 4

Running time of Kruskal's algorithm

10 March.

First, sorting all edges by weight takes $O(m \log_2 m)$ steps. Then, iterate through all edges, if $C_u = C_v$, do nothing, and this steps take $O(1)$ steps. If not, then add edges and update components, which takes $O(n)$ steps, and will run this case $n - 1$ times. Thus, the total time:

$$O(m \log_2 m) + O(m) + O\left(n^2\right) = O(m \log_2 m + n^2) = \begin{cases} O(m \log m), & \text{if } m = \Omega\left(\dfrac{n^2}{\log n}\right); \\ O\left(n^2\right), & \text{if } m = O\left(\dfrac{n^2}{\log n}\right). \end{cases}$$

With more improvement,

$$O(m \log m) \text{ for all } m.$$

2.2 Dijkstra's Algorithm

Drawbacks of trees as networks

(1) Unique path between any two vertices, which could be unnecessarily long.

(2) No redundancy/robustness: removing any one edge/vertex disconnects the network.

Question. Given a network $G = (V, E)$ and weights $w : E \to \mathbb{R}_{\geq 0} \cup \{\infty\}$, and given two vertices $u, v \in V$, how do we find the cheapest path from u to v?

Observation. We may assume G is complete. If an edge $\{x, y\}$ is missing, add it with ∞ weight.

Lemma 2.2.1. If P is a cheapest path from u to v, then for any $x, y \in P$, $P[x, y]$ is a shortest path from x to y.

Algorithm 2.4: Dijkstra's algorithm

Data: A complete graph $G = (V, E)$.
Result: For any vertex x, the cheapest graph from u to x.
Idea: Keep a set W of vertices which we have already found the cheapest path from u. For other vertices, keep track of a tentative cheapest path, i.e. the cheapest path we have found so far.

```
W ← {u}, t(u) ← 0, prev(u) ← ∅.
for x ∈ V \ {u} do
    t(x) ← w(u, x).
    prev(x) ← u.
while W ≠ V do
    Let y = arg min_{x∈V\W} t(x), i.e. t(y) minimizes t(x) for all x ∉ W.
    Add y to W.
    for every x ∉ W ∪ {y} do
        if t(x) > t(y) + w({y, x}) then
            t(x) ← t(y) + w({y, x}), prev(x) = y.
Cheapest path from x to u is x, prev(x), prev(prev(x)), ..., u with cost t(x).
```

Theorem 2.2.1. Dijkstra's algorithm terminates after $n - 1$ rounds and returns shortest path.

Proof. Each round adds a new vertex to W, which never gets removed, so after $n - 1$ rounds, $W = V$. For correctness, we can do induction on the number of rounds:

Claim 2.2.1. At the beginning of the i-th iterartion:

(a) $|W| = i$.

(b) for every $x \in V$, the vertices $x, \text{prev}(x), \text{prev}(\text{prev}(x)), \ldots, u$ form a path from x to u of weight $t(x)$, where all vertices except x lie in W.

(c) for any vertex $z \in V \setminus W$ with $t(z) = \min\{t(x) : x \in V \setminus W\}$, $t(z)$ is the minimum cost of a uz-path.

(d) For every $y \in V \setminus W$ and $v \in W$,

$$t(y) \leq t(v) + w(\{v, y\}).$$

Proof.

- Base case: $i = 1$, (a), (b), (d) are obvious. For (c), suppose not, then if z minimize $t(\cdot)$ but there is a cheaper uz-path, say

$$u \to x \to \cdots \to z \in P,$$

then

$$w(P) = w(\{u, x\}) + w(\{x, \dots\}) + \cdots \geq w(\{u, x\}) = t(x) \geq t(z),$$

which is a contradiction.

- Induction step: (a) is obvious.

Now we show (b). If we didn't update $t(x), \text{prev}(x)$ in the $(i-1)$-th iteration, then this is true by induction. If we did update, then $\text{prev}(x) = v_0$ where v_0 is the vertex added to W in the $(i-1)$-th iteration, and $t(x) = t(v_0) + w(\{v_0, x\}) =$ weight of path $u \to \cdots \to v_0 \to x$, and u to $\text{prev}(v_0)$ all in W by induction, and v_0 was added to W in the $(i-1)$-th iteration.

Now we show (c). Suppose not, then $\exists z \in V \setminus W$ minimizing $t(z)$ but $t(z) > d(z, u)$. Consider the cheapest path from u to z, say P. Let y be the first vertex in P (starting from u) that is not in W. (such y exists since z is not in W) Let x be the preceding vertex on P, noting that $x \in W$. Then,

$$t(z) > w(P) = w(P_{[u,x]}) + w(\{x, y\}) + w(P_{[y,z]}) \geq w(P_{[u,x]}) + w(\{x, y\}).$$

By Lemma 2.2.1, $P_{[u,x]}$ has to be a lightest path from u to x. By induction (where we add x to W), $t(x) = w(P_{[u,x]})$. Thus,

$$t(z) > w(P_{[u,x]}) + w(\{x, y\}) = t(x) + w(\{x, y\}) \geq t(y),$$

which is by the induction statement of (d). However, this contradicts z minimizeing $t(v)$ for $v \in V \setminus W$.

As for (d), after the step where we added v to W, we check whether $t(v) + w(\{v, y\}) < t(y)$, and update if so. Hence, after this round, $t(v)$ will not change, and $t(v)$ is decreasing, so it is impossible that $t(y) > t(v) + w(\{v, y\})$.

⊛

This implies the correctness: for any x, at the iteration when we add x to W, (c) implies we have the correct path for x and we never change it subsequently. ■

Lecture 5

2.2.1 Running Time of Dijkstra's Algorithm

13 March.

Since we have $n - 1$ iterations, and in each iteration, we have to find the minimum, which takes $O(n)$ steps, and updating also takes $O(n)$ steps, so the total running time is $O\left(n^2\right)$.

2.3 Traveling Salesman Problem

Problem 2.3.1 (Traveling Salesman Problem). If we have a weighted graph $G = (V, E)$, where each vertex is a city and each edge is a road between two cities, and we are at $v \in V$ initially. Then, how to travel to every city exactly once and then return to the starting point v while minimizing the cost?

Definition 2.3.1 (Hamilton cycle)**.** A cycle that goes through every vertex in a graph is called a Hamilton cycle.

Remark 2.3.1. Traveling Salesman Problem's goal is to find a Hamilton cycle of minimum cost of the given graph.

Observation. The number of possible routes is $\frac{1}{2}(n-1)!$ since we can just permutate the $n-1$ vertices that is not the starting point, and note that the direction is not important, so $u \to a \to b \to u$ and $u \to b \to a \to u$ is considered same routes.

Thus, the brute force takes $O(n) \cdot \frac{1}{2}(n-1)! = O(n!)$ time since for each route we have to go through this path to compute the cost.

Remark 2.3.2. The Held-Karp Algorithm, which uses dynamic programming, takes $O\left(n^2 2^n\right)$ time to solve Traveling Salesman Problem (also exponential in memory).

Question (Open question)**.**

- Is there a polynomial time algorithm?
- Is there an algorithm takes $O(1.99999999^n)$ time?

Chapter 3

Matchings

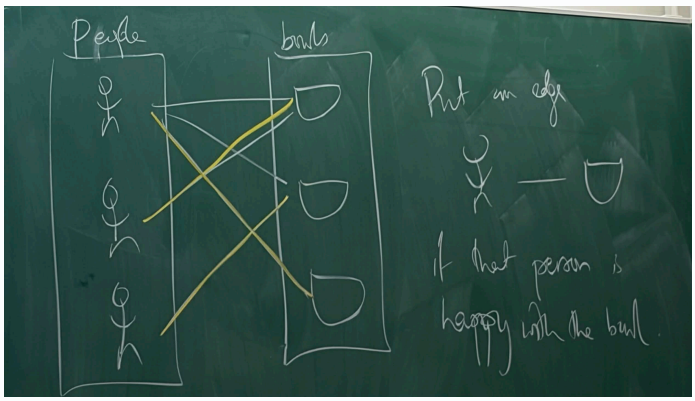
Lecture 6

3.1 Hall and Tutte and Dirac

17 March.

Example 3.1.1. n friends going for dinner. n bowls of beef noodle soup served. Each person has a subset of the bowls they would be happy with. Can we give one bowl to each person and making every one happy?

Proof. We can build a graph as follow:



⊛

Definition 3.1.1. A graph G is bipartite if we can partition $V(G) = X \uplus Y$ s.t. for every $e \in E(G)$,

$$|e \cap X| = 1 = |e \cap Y|.$$

Definition 3.1.2. A matching in a graph is a set of pairwise-disjoint edges. If a vertex is in an edge of the matching, it is called "saturated". If every vertex is saturated, the matching is perfect.

Observation. If there is some $S \subseteq X$ with $|N(S)| < |S|$, then there is no matching saturating X.

Proof. If there is a matching saturating X and $\exists S \subseteq X$ with $|N(S)| < |S|$, then each vertex in S would match with some vertex in $N(S)$ in the matching saturating X. Note that every vertex in S cannot match to same vertex. Hence, $|N(S)| \geq |S|$, which is a contradiction. ■

Theorem 3.1.1 (Hall's Marriage Theorem). Let $G = (X \cup Y, E)$ be a bipartite graph. If $|N(S)| \geq |S|$ for all $S \subseteq X$, then G has a matching saturating X.

Proof. We can do induction on $|X|$.

- Base case: $|X| = 1$. Let $S = X$, and $|N(X)| \geq |X| = 1$, then there exists $y \in Y$ s.t. $\{x, y\} \in E$, so we can pick this edge to form a matching saturating X.
- Induction Step: $|X| \geq 2$.
  - Case 1: $\forall \varnothing \neq S \subsetneq X$, $|N(S)| > |S|$. Let $\{x, y\}$ be an arbitrary edge, where $x \in X$ and $y \in Y$. Let $G' = (X' \cup Y', E')$ where $X' = X - \{x\}$, $Y' = Y - \{y\}$, and $E' = E|_{X' \times Y'}$. We use induction to find a matching M' in G' saturating X'. Then, $M = M' \cup \{x, y\}$ is a matching saturating X. Note that we can use induction hypothesis since for $S \subseteq X'$, $N_{G'}(S) = N_G(S) \setminus \{y\}$. Hence,
  $$|N_{G'}(S)| \geq |N_G(S)| - 1 \geq |S|,$$
  and thus we can find M'.
  - Case 2: there exists some $\varnothing \neq X_1 \subsetneq X$ with $|N(X_1)| = |X_1|$. Let $N(X_1) = Y_1$. Let $X_2 = X \setminus X_1$, and $Y_2 = Y \setminus Y_1$, and $G_1 = G[X_1 \cup Y_1]$. By induction, G_1 has a matching M_1 saturating X_1 since for all $S \subseteq X_1$, we have $S \subseteq X$ and thus
  $$|N_G(S)| \geq |S|,$$
  and note that $N_{G_1}(S) = N_G(S)$ since $N_G(S) \subseteq N_G(X_1) = Y_1$. If we can find a matching M_2 in $G_2 = G[X_2 \cup Y_2]$, then $M = M_1 \cup M_2$ is a matching saturating X. Let $S \subseteq X_2$. Consider $S' = S \cup X_1$. By assumption, $|N_G(S')| \geq |S'|$. Note that $|S'| = |S| + |X_1|$. Hence,
  $$N_G(S') = N_G(S \cup X_1) = Y_1 \cup \underbrace{N_{G_2}(S)}_{\subseteq Y_2}.$$
  Note that $N_G(S') = Y_1 \cup N_{G_2}(S)$ is a disjoint union since for $N_G(S)$, it is either in Y_1 or not, if not, then it is in $N_{G_2}(S)$. This shows
  $$|S| + |X_1| = |S'| \leq |N_G(S')| = |Y_1| + |N_{G_2}(S)| = |X_1| + |N_{G_2}(S)|.$$
  Hence, $|N_{G_2}(S)| \geq |S|$ for all $S \subseteq X_2$. By induction, we can find a matching M_2 in G_2 saturating X_2.

■

Corollary 3.1.1. If G is a d-regular bipartite graph, then G has a perfect matching.

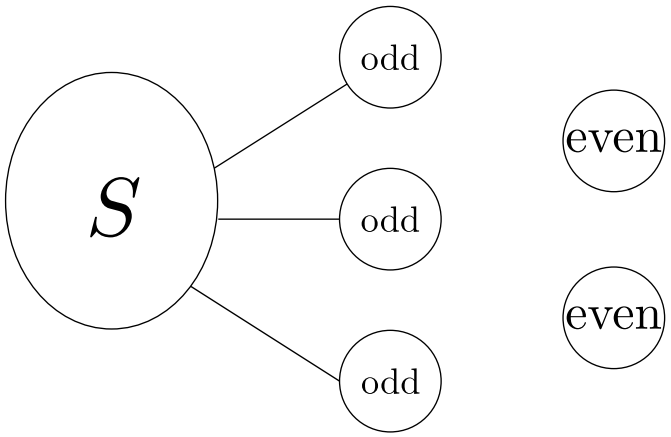
Proof. Given any $S \subseteq X$,
$$d|S| = \#\{\text{edges between } S, N(S)\} \leq d|N(S)|$$
since for vertices in $N(S)$, they may be adjacent to vertex not in S, so by Hall's theorem, there is a matching saturating X, which is a perfect matching since
$$d|X| = \#\{\text{edges between } X, Y\} = d|Y|,$$
and thus $|X| = |Y|$. ■

Question. What are necessary and sufficient conditions to have a perfect matching in general (i.e. non-bipartite) graph?

For necessary condition:

- n should be even.
- every vertex should has positive degree.
- every connected components should be even.
- for any $S \subseteq V(G)$, $o(G-S) \leq |S|$, where $o(G-S)$ is the number of odd components in $G-S$.

Remark 3.1.1. For the last argument, it is because G has a perfect matching, and for the odd components in $G-S$, there must be edges between them and S, i.e. some vertices in those odd componenets must match with the vertex in S, and thus $o(G-S) \leq |S|$.



Theorem 3.1.2 (Tutte, 1947). If $o(G-S) \leq |S|$ for every $S \subseteq V(G)$, then G has a perfect matching.

proof (Lovasz, 1975). Note that for $S = \varnothing$, we have $o(G) \leq |0|$, so $|V(G)|$ is even. If G is complete, then G obviously has a perfect matching. Now we consider the case that G is not complete. Suppose for contradiction. G is a counterexample on n vertices with the maximum number of edges.

Claim 3.1.1. $\forall \{x,y\} \notin E(G)$, $G' = G + \{x,y\}$ has a perfect matching.

Proof. We have to show G' satisfies Tutte's condition. If so, then since G was a longest counterexample, G' cannot be a counterexample, so G' has a perfect matching.

Note that we have $o(G-S) \leq |S|$ for all $S \subseteq V(G)$. If $\{x,y\} \cap S \neq \varnothing$, then $o(G'-S) = o(G-S)$. If $\{x,y\}$ is contained in a component of $G-S$, components don't change, so $o(G'-S) \leq |S|$. If $\{x,y\}$ connects two components of $G-S$, then no matter what happens,

$$o(G'-S) \leq o(G-S) \leq |S|.$$

⊛

Remark 3.1.2. The perfect matching must pick $\{x,y\}$, otherwise we have a perfect matching in G.

Let $U = \{v \in G : \deg(v) = n-1\}$.

- Case 1: Every component of $G-U$ is a clique(complete subgraph). Each even clique admits a perfect matching. Each odd clique has a matching with only one unsaturated vertex. By Tutte's condition, number of odd cliques $\leq |U|$, say there are k odd cliques. Thus, we can match the unsaturated vertices to different vertices in U. Note that there will be even number

of unused vertices in U since

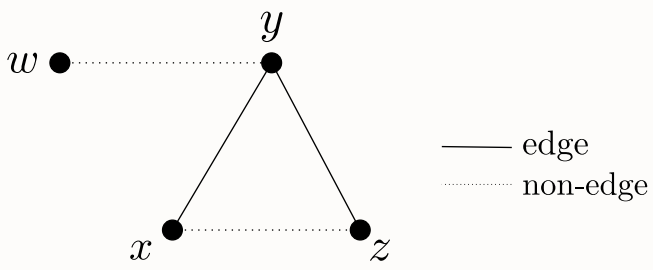
$$|V(G)| = |U| + \sum \text{size of components},$$

and since $|V(G)|$ is even, we have

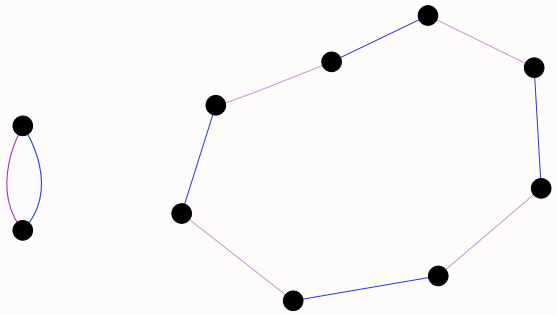
$$|U| \equiv k \mod 2.$$

This shows $|U| - k$ is even, i.e. after we match the unsaturated vertex in each odd component to U, the number of unused verteices in U is even, and thus we can match them arbitrarily.

- Case 2: There is a non-clique component in $G - U$. We can find the following configuration:



The reason is in $G - U$, there must be x, y, z which do not form a triangle, otherwise every component is a clique. On the other hand, $y \notin U$, so there exists some w that is not adjacent to y. Let M_1 be a perfect matching in $G + \{w, y\}$ and let M_2 be a perfect matching in $G + \{x, z\}$. Let $G' = M_1 \cup M_2$. Then the components of G' are either isolated edges (really in G) or even edges, alternating M_1 and M_2 edges.



To build a matching M of G, we can take all edges that are components in G'. Consider the cycle component C in G'. If the edges $\{y, w\}$ is not in C, then take the M_1 edges. If the edge $\{x, z\}$ is not in C, then take the M_2 edges. If both edges are in C: If we delete y from this cycle, then there remains a path with an even number of edges and odd number of vertices. We can delete one of x or z s.t. the two remaining paths both have an even number of vertices. Note that we have $\{y, z\}, \{y, x\}$ these two edges, so we can take the edges incident to the deleted pair and since the remaining two subpaths are of even vertices, so we can take alternating edges on two subpaths to form a matching.

■

Remark 3.1.3. Hence, in fact, this is an if and only if statement: $o(G-S) \leq |S|$ for every $S \subseteq V(G)$ if and only if G has a perfect matching.

Lecture 7

20 March.

Theorem 3.1.3 (Dirac, 1952). Let G be an n-vertex graph with minimum degree at least $\frac{n}{2}$, then G has a Hamilton cycle.

proof of Dirac's theorem. We first introduce a lemma.

Lemma 3.1.1. Let G be an n-vertex graph with minimum degree $\geq \frac{n}{2}$. Let P be a longest path in G, then

(1) If u, v are the endpoint of P, then $N(u) \cup N(v) \subseteq V(P)$.

(2) There is a cycle C with $V(C) = V(P)$.

Proof.

(1) If an endpoint has a neighbor not in P, we can extend P by using the edge incident to that endpoint and that neighbor, which is a contradiction.

(2) If $\{u, v\} \in E$, add it to the path to get a cycle. Otherwise, let

$$P = u = v_1 \to v_2 \to v_3 \to v_4 \to \cdots \to v_i \to v_{i+1} \to \cdots \to v_m = v.$$

If $\exists i$ s.t. $\{v_i, v\}, \{u, v_{i+1}\} \in E$, then take C to be

$$u \to v_{i+1} \to v_{i+2} \to \cdots \to v_{m-1} \to v \to v_i \to v_{i-1} \to \cdots \to v_2 \to u.$$

Such an i must exist: let $S = \{v_i : \{u, v_{i+1}\} \in E\}$, then $|S| = |N(u)| = \deg(u) \geq \frac{n}{2}$. Also, $|N(v)| = \deg(v) \geq \frac{n}{2}$ and $N(u) \cup N(v) \subseteq V(P)$. Note that

$$|S \cup N(v)| \leq |\{u, v_2, \ldots, v_{m-1}\}| \leq n - 1.$$

Thus, $S \cap N(v) \neq \varnothing$.

■

If we have this lemma, then let P be a longest path and v is an endpoint. By (1), $N(v) \subseteq N(P)$. Hence, $|V(P)| \geq \frac{n}{2} + 1$(contain all neighbors of V and v itself). By (2), there is a cycle C with $V(C) = V(P)$. If $V(P) = V(G)$, then C is a Hamilton cycle. Otherwise, let $x \in V(G) \setminus V(C)$, since $|V(C)| = |V(P)| > \frac{n}{2}$, and $N(x) \geq \frac{n}{2}$ and $|V(C) \cup N(x)| \leq n$, so $N(x) \cap V(C) \neq \varnothing$, i.e. there is a neighborhood of x in C. Then we find a path longer than P, contradicting our choice of P. ■

Remark 3.1.4. This minimum degree condition is sufficient but not necessary, e.g. $G = C_n, n \geq 5$.

Remark 3.1.5. The minimum degree condition is best possible. Otherwise, consider a bipartite graph $G = (U \cup V, E)$, where $|U| = \frac{n}{2} - 1$ and $|V| = \frac{n}{2} + 1$, and for each $u \in U$ and $v \in V$ we have $\{u, v\} \in E$. Thus, this subgraph can not have a Hamilton cycle.

Theorem 3.1.4 (Ore). Let G be an n-vertex graph, such that whenever $\{u, v\} \notin E(G)$, we have $\deg(u) + \deg(v) \geq n$, then G has a Hamilton cycle.

Lecture 8

24 March.

Observation. Let $G = (V, E)$ and suppose G has a hamilton cycle. Then, for every $\varnothing \neq S \subseteq V$,

$$\underbrace{c(G - S)}_{\#\text{ of cpts}} \leq |S|$$

Let C be a Hamiltonian cycle of G. Consider removing the vertices in S from C. Since C is a cycle passing through all vertices of G, deleting S breaks C into a collection of vertex-disjoint paths, each of which lies entirely in $G - S$.

Each such path is contained in a connected component of $G - S$. Moreover, every component of $G - S$ must contain at least one of these path segments, since C visits every vertex of G.

Now fix a component H of $G - S$. Along the cycle C, the vertices of H appear consecutively in one or more segments. Each maximal segment of C contained in H must be entered from S and exited back to S. Therefore, for each such segment, there are at least two edges of C between S and H (one entering and one leaving).

In particular, each component H of $G - S$ is incident with at least two edges of C that go between S and $V(H)$. Hence, the total number of edges of C between S and $G - S$ is at least

$$2\, c(G - S).$$

On the other hand, each vertex in S has degree exactly 2 in the cycle C. Therefore, the total number of edges of C incident to vertices in S is at most

$$2|S|.$$

All edges between S and $G - S$ are among these edges, so

$$\#\{\text{edges between } S \text{ and } G - S \text{ in } C\} \leq 2|S|.$$

Combining the two inequalities, we obtain

$$2\, c(G - S) \leq 2|S|,$$

which implies

$$c(G - S) \leq |S|.$$

Definition 3.1.3 (toughness). Given $t \in \mathbb{R}$, we say a graph G is t-tough if

$$|S| \geq t c(G - S)$$

for all $S \subseteq V(G)$ and $S \neq \varnothing$.

Hence, the observation above states that the Hamiltonicity implies 1-tough.

Conjecture 3.1.1 (Chvatal, 1973). There is some t s.t. t-tough implies Hamiltonicity.

Construction of Bauer-Broersma-Veldman (2000): families of non-Hamiltonian graphs with toughness $\left(\frac{9}{4} - o(1)\right)$.

Note 3.1.1. Toughness of $C_n \coloneqq 1$.

3.2 Complexity

Definition 3.2.1. A decision problem is a problem of answer yes/no.

Remark 3.2.1. More formally, a decision problem can be viewed as a language $L \subseteq \{0,1\}^*$, where an input x is a **yes-instance** if $x \in L$, and a **no-instance** otherwise.

Definition 3.2.2 (polynomial time)**.** A decision problem with an algorithm to find the answer in polynomial time in the size of the input means there exists a constant α s.t. if input has size n, then we can find the answer in time $O(n^\alpha)$.

Remark 3.2.2. The class of all decision problems solvable in polynomial time is denoted by P. Polynomial time is considered "efficient" or "tractable" in theoretical computer science.

Definition 3.2.3 (Nondeterministic polynomial time)**.** "Yes" answers come with a certificate that can be verified in polynomial time.

Remark 3.2.3. More precisely, a decision problem is in NP if there exists a polynomial-time algorithm $V(x,c)$ (called a **verifier**) such that:

- if x is a yes-instance, then there exists a certificate c with $|c| \leq \text{poly}(|x|)$ such that $V(x,c) = 1$,
- if x is a no-instance, then for all c, $V(x,c) = 0$.

Example 3.2.1. In Hamiltonicity problem, the certificate is the hamilton cycle.

Remark 3.2.4. Given a sequence of vertices, we can check in polynomial time whether it forms a Hamiltonian cycle by verifying:

- all vertices appear exactly once,
- consecutive vertices are adjacent,
- the first and last vertices are also adjacent.

Claim 3.2.1. $P \subseteq NP$.

Proof. To check the certificate, ignore it and solve the problem from scratch. ⊛

Remark 3.2.5. Indeed, if a problem can be solved in polynomial time, then we can verify it in polynomial time by simply recomputing the answer. Hence every problem in P is also in NP.

Definition 3.2.4 (co-NP)**.** co-NP problem is the class of decision problems where No instances have a certificate that can be verified in polynomial time.

Remark 3.2.6. Equivalently, a problem L is in co-NP if its complement $\overline{L}$ is in NP.

Example 3.2.2 (co-NP)**.**

- anything in P.
- Is this graph not Hamiltonian?
- Does a graph G has a perfect matching? (This problem is in $NP \cap$ co-NP since for yes instances the certificate is the perfect matching itself and for no instances we can use Tutte's theorem, i.e. the certificate is $S \subseteq V$ s.t. $o(G - S) > |S|$).

Remark 3.2.7. For many problems (such as Hamiltonicity), it is unknown whether they belong to co-NP. In particular, we do not know whether Hamiltonicity has efficiently verifiable certificates for *non*-existence.

Question. Does $P = NP$?

Common belief: $P \neq NP$.

Remark 3.2.8. This is one of the most important open problems in computer science. It asks whether every efficiently verifiable problem is also efficiently solvable.

Definition 3.2.5 (NP-hard)**.** A problem that any NP-problem can be reduced to in polynomial time is called NP-hard.

Remark 3.2.9. More precisely, a problem H is NP-hard if for every problem $L \in NP$, there exists a polynomial-time reduction $L \leq_p H$. Note that NP-hard problems are not required to be in NP (they may even be undecidable).

Definition 3.2.6 (NP-complete)**.** NP-complete = NP-hard $\cap NP$.

Remark 3.2.10. NP-complete problems are the "hardest" problems in NP. If any NP-complete problem can be solved in polynomial time, then $P = NP$.

Karp (1972) provided an initial list of many NP-complete problem, including

- Hamiltonicity.
- 3-Colorability.
- 3-SAT.

Remark 3.2.11. These problems are polynomially equivalent under reductions, meaning that an efficient algorithm for any one of them would yield efficient algorithms for all NP problems.

3.3 Maximum matching problem

Question. Given a graph G, what is the size of its largest matching?

This is an optimization problem (OPT).
The related decision problem is: Is the size of the largest matching $\geq k$?

Theorem 3.3.1 (Hall, min-max version)**.** Let $G = (X \cup Y, E)$ be a bipartite graph. Then,

$$\max |M| = \min_{S \subseteq X} |X| - |S| + |N(S)|,$$

where $|M|$ is the number of edges in a matching.

Proof. We first show

$$\max |M| \leq \min_{S \subseteq X} \big(|X| - |S| + |N(S)|\big).$$

Fix any $S \subseteq X$, and let M be any matching in G.

Each vertex in $N(S)$ can be matched to at most one vertex in S, so at most $|N(S)|$ vertices of S are saturated by M. Hence at least

$$|S| - |N(S)|$$

vertices of S are unsaturated.

Therefore, the total number of vertices in X that are saturated by M is at most

$$|X| - (|S| - |N(S)|) = |X| - |S| + |N(S)|.$$

Since this holds for every $S \subseteq X$, we obtain

$$|M| \leq \min_{S \subseteq X} \big(|X| - |S| + |N(S)|\big).$$

Taking maximum over all matchings M, we get

$$\max |M| \leq \min_{S \subseteq X} \big(|X| - |S| + |N(S)|\big).$$

Now we prove the reverse inequality.

Define the *deficiency* of a set $S \subseteq X$ by

$$|S| - |N(S)|.$$

Let

$$t := \max_{S \subseteq X} |S| - |N(S)| = \max_{S \subseteq X} \big(|S| - |N(S)|\big).$$

Then for every $S \subseteq X$, we have

$$|S| - |N(S)| \leq t,$$

which implies

$$|N(S)| \geq |S| - t.$$

We now construct a new bipartite graph $G' = (X \cup (Y \cup Y'), E')$ by adding a set Y' of t new vertices, each adjacent to every vertex in X.

For any nonempty $S \subseteq X$, we have

$$N_{G'}(S) = N_G(S) \cup Y',$$

and hence

$$|N_{G'}(S)| = |N_G(S)| + t \geq (|S| - t) + t = |S|.$$

Thus Hall's condition holds in G', and by Hall's theorem there exists a matching in G' that saturates all vertices in X.

Since $|Y'| = t$, at most t vertices of X are matched to vertices in Y'. Therefore, at least $|X| - t$ vertices of X are matched to vertices in Y.

Removing all edges incident to Y', we obtain a matching in G of size at least $|X| - t$. Hence

$$\max |M| \geq |X| - t.$$

Finally, observe that

$$|X| - t = |X| - \max_{S}(|S| - |N(S)|) = \min_{S} \big(|X| - |S| + |N(S)|\big).$$

Therefore,

$$\max |M| \geq \min_{S \subseteq X} \big(|X| - |S| + |N(S)|\big).$$

Combining the two inequalities, we conclude

$$\max |M| = \min_{S \subseteq X} \big(|X| - |S| + |N(S)|\big).$$

■

Remark 3.3.1. This shows that "Does a bipartite graph have a matching of size k?" is in NP and co-NP.

Definition 3.3.1. A vertex cover of a graph is a set $C \subseteq V$ of vertices such that for every edge $e \in E$, $e \cap C \neq \varnothing$.

Definition 3.3.2. Given a graph G, we write

$$\alpha'(G) = \max\{|M| : M \subseteq E \text{ is a matching}\}$$
$$\beta(G) = \min\{|C| : C \subseteq V \text{ is a vertex cover}\}.$$

Claim 3.3.1. For any G, $\alpha'(G) \leq \beta(G)$, i.e. for any matching M and vertex cover C, $|M| \leq |C|$.

Proof. Let M be any matching in G, and let C be any vertex cover of G.

By definition, every edge in M must be incident to at least one vertex in C, since C is a vertex cover.

On the other hand, since M is a matching, no two edges in M share a common endpoint. Therefore, each vertex in C can cover at most one edge of M.

Hence, to cover all edges in M, we need at least $|M|$ vertices in C. That is,

$$|C| \geq |M|.$$

Since this holds for all matchings M and all vertex covers C, we conclude

$$\alpha'(G) \leq \beta(G).$$

⊛

Remark 3.3.2. We don't necessarily have $\alpha'(G) = \beta(G)$ (consider K_3).

Theorem 3.3.2 (Konig). If $G = (X \cup Y, E)$ is bipartite, then

$$\alpha'(G) = \beta(G).$$

Proof. We already know $\alpha'(G) \leq \beta(G)$. Thus, we need to show $\alpha'(G) \geq \beta(G)$. Let $C \subseteq V$ be a smallest vertex cover, then we need to find a matching of size $|C|$. We will use Hall's theorem to find a matching in $(C \cap X) \cup (Y \setminus C)$ saturating $C \cap X$ and a matching in $(C \cap Y) \cup (X \setminus C)$ saturating $C \cap Y$. Then, the union of these matchings is a matching of size

$$|C \cap X| + |C \cap Y| = |C|.$$

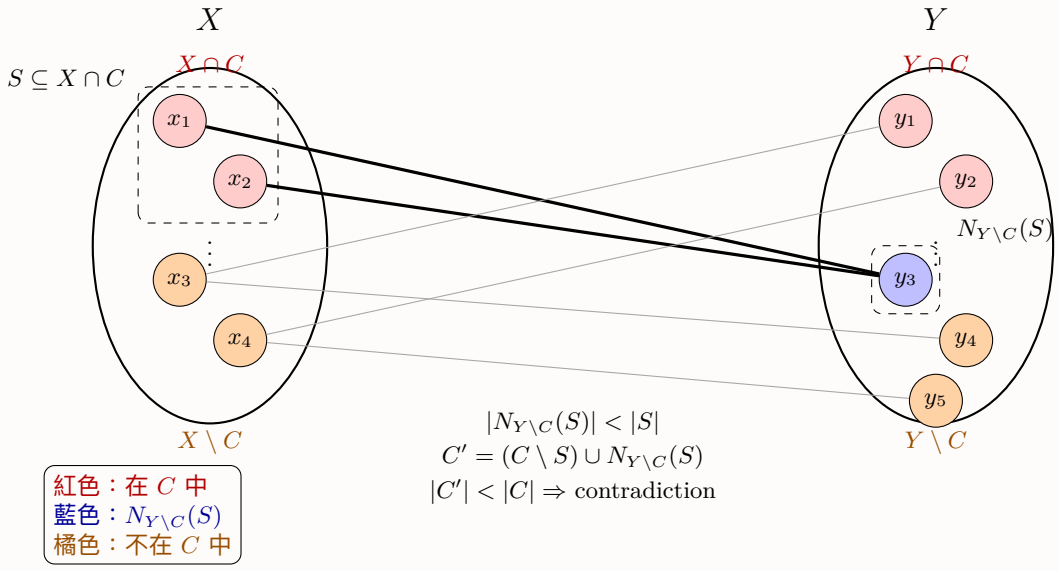
If not, then WLOG there is no matching in $(X \cap C) \cup (Y \setminus C)$ saturating $X \cap C$. By Hall's theorem, $\exists S \subseteq X \cap C$ s.t.

$$|N_{Y \setminus C}(S)| < |S|,$$

but then

$$C' = (C \setminus S) \cup N_{Y \setminus C}(S)$$

is a vertex cover of smaller size, which is a contradiction.



■

Lecture 9

27 March.

Theorem 3.3.3 (Tutte–Berge min–max formula)**.** For every finite graph $G = (V, E)$,

$$2\alpha'(G) = \min_{S \subseteq V} \Big(|V| + |S| - o(G - S) \Big),$$

where $\alpha'(G)$ is the maximum size of a matching in G, and $o(G-S)$ is the number of odd components of $G - S$.

Proof. Set $n := |V|$, and define the *deficiency*

$$\mathrm{def}(G) := \max_{S \subseteq V} \big(o(G - S) - |S| \big).$$

The claimed identity is equivalent to

$$\alpha'(G) = \frac{n - \mathrm{def}(G)}{2}.$$

First, we show $n - 2\alpha'(G) \geq \mathrm{def}(G)$. Let M be any matching. Fix $S \subseteq V$. Consider odd components of $G - S$. In an odd component C, edges of M internal to C cover an even number of vertices; therefore at least one vertex of C is either unmatched by M or matched by an edge going from C to S. Each vertex of S is incident to at most one matching edge, so at most $|S|$ odd components can be "accounted for" by such edges to S. Hence at least $o(G - S) - |S|$ odd components contain a vertex unmatched by M. So the total number of unmatched vertices, namely $n - 2|M|$, satisfies

$$n - 2|M| \geq o(G - S) - |S|.$$

Since this holds for every S, we get

$$n - 2|M| \geq \mathrm{def}(G).$$

Taking $|M| = \alpha'(G)$ gives

$$n - 2\alpha'(G) \geq \mathrm{def}(G). \tag{1}$$

For the reverse inequality, let $d := \mathrm{def}(G)$. By parity, $d \equiv n \pmod 2$ (because $o(G-S) \equiv |V \setminus S| \bmod 2$ for every S). Hence $n + d$ is even.

Now form the graph

$$H := G \vee K_d$$

(join G with a complete graph on d new vertices). We claim H has a perfect matching. By Tutte's 1-factor theorem, it is enough to show

$$o(H-T) \le |T| \quad \text{for all } T \subseteq V(H).$$

Write $T = S \cup R$, where $S \subseteq V(G)$ and $R \subseteq V(K_d)$.

If $R \neq V(K_d)$, then at least one vertex of K_d remains in $H - T$; because that vertex is adjacent to every remaining vertex of G, all vertices of $H - T$ lie in one connected component. Thus $o(H-T) \le 1 \le |T|$ unless $T = \varnothing$; and for $T = \varnothing$, $|V(H)| = n + d$ is even, so $o(H) = 0$.

If $R = V(K_d)$, then $H - T = G - S$, so

$$o(H-T) = o(G-S) \le |S| + d = |T|,$$

because $d = \max_{X \subseteq V}(o(G-X) - |X|)$.

Therefore Tutte's condition holds, and H has a perfect matching P. Restrict P to edges of G: this gives a matching M in G. At most d vertices of G can be matched in P to vertices of K_d, so at least $n - d$ vertices of G are matched within G. Hence

$$|M| \ge \frac{n-d}{2},$$

so

$$\alpha'(G) \ge \frac{n-d}{2} \quad \Longrightarrow \quad n - 2\alpha'(G) \le d = \operatorname{def}(G). \tag{2}$$

Combining (1) and (2):

$$n - 2\alpha'(G) = \operatorname{def}(G).$$

Rewriting,

$$2\alpha'(G) = n - \max_{S \subseteq V}\big(o(G-S) - |S|\big) = \min_{S \subseteq V}\big(n + |S| - o(G-S)\big),$$

which is exactly the formula.

thick red edges = the perfect matching P

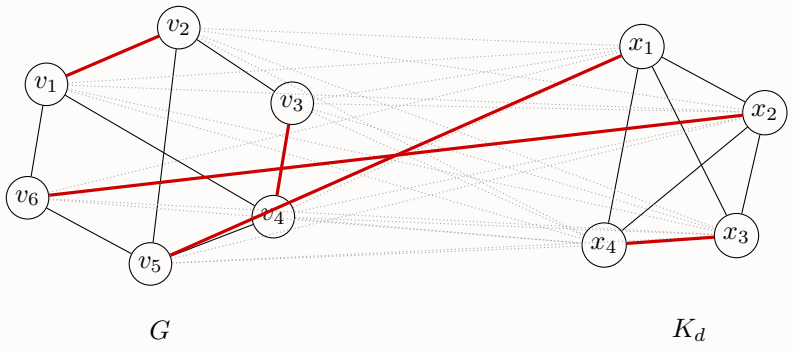


Figure 3.1: $H = G \vee K_d$.

■

3.4 Traveling salesman problem

Suppose we have $w : E(K_n) \to \mathbb{R}_{\ge 0}$, and our goal is to find a Hamilton cycle of minimal weight. Thus, we have to solve the following question:

Question. Does there exist a Hamilton cycle of weight $\le C$?

This problem is NP-complete since we know "Is there a Hamilton cycle in G?" is NP-complete and it suffices to reduce this to TSP.

The reduction can be implemented by taking $V(K_n) = V(G)$ and

$$w(e) = \begin{cases} 0, & \text{if } e \in E(G); \\ 1, & \text{if } e \notin E(G). \end{cases}$$

Definition 3.4.1 (Approximation algorithm)**.** Given any optimization problem, a c-approximation algorithm is an algorithm that returns a solution guaranteed to be within a factor $c \in \mathbb{R}$ of the optimum.

We say an edge weighting of K_n satisfies the triangle inequality if

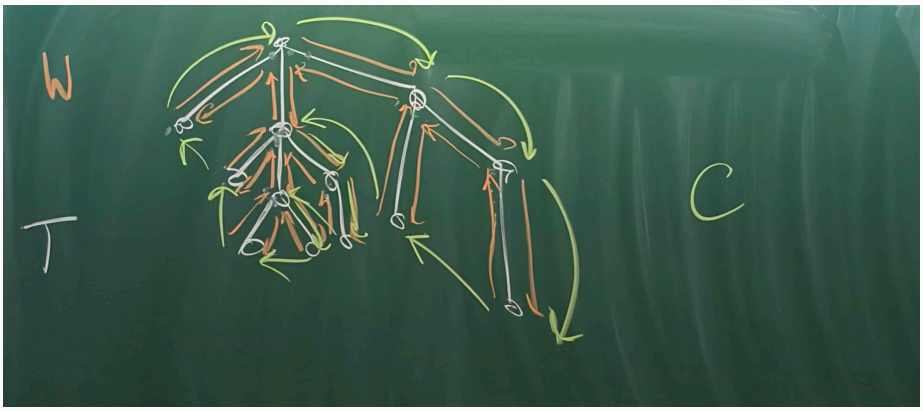
$$\forall x, y \in V(K_n), \quad w(\{x, y\}) + w(\{y, z\}) \geq w(\{x, z\}).$$

Theorem 3.4.1. If w satisfies the triangle inequality. Then there is a polynomial time 2-approximation algorithm of TSP.

Proof. We can first find a minimum spanning tree T. Walk around the edge of the tree, traversing each edge in both direction. Suppose the cycle is W, then $w(W) = 2w(T)$.

Then, we can build a cycle by taking shortcuts: every time W sends us to a vertex we've already visited, skip ahead to the next, and thus we can form another cycle C, while the triangle inequality states that $w(C) \leq w(W) = 2w(T)$. Let C^* be the optimal cycle, we wanted $w(C) \leq 2w(C^*)$. Removing any edge from C^* gives a Hamilton path P which is a spanning tree. Thus,

$$w(C^*) \geq w(P) \geq w(T) \Rightarrow w(C^*) \leq w(C) \leq 2w(T) \leq 2w(C^*).$$



Hence,

$$w(C) \leq 2w(C^*).$$

■

3.5 Maximum matching

Question. How do we find a maximum matching?

Remark 3.5.1. Maximum is not equal to maximal, where maximum means largest there is, and maximal means cannot be extended.

Claim 3.5.1. There exists a simple $\frac{1}{2}$-approximation algorithm for the maximum matching problem.

Proof. Consider the following greedy algorithm: process the edges of G in an arbitrary order, and add an edge to the matching M if it is disjoint from all previously selected edges. The resulting matching M is *maximal*.

Let M^* be a maximum matching. We claim that

$$|M^*| \leq 2|M|.$$

Since M is maximal, no edge in $E(G) \setminus M$ can be added to M. In particular, for every edge $e = (u, v) \in M^*$, at least one of its endpoints must already be saturated by M; otherwise, e could be added to M, contradicting maximality.

Thus, for every edge $e \in M^*$, we can choose one endpoint that is saturated by M. Define a mapping

$$f : M^* \to V(M)$$

by assigning each edge $e \in M^*$ to one of its endpoints that is saturated by M.

We now show that f is injective. Suppose $e_1, e_2 \in M^*$ are distinct edges such that

$$f(e_1) = f(e_2) = v.$$

Then both e_1 and e_2 are incident to v, which contradicts the fact that M^* is a matching. Hence, f is injective.

Therefore,

$$|M^*| \leq |V(M)|.$$

Since every edge in M saturates exactly two vertices and no two edges share a vertex,

$$|V(M)| = 2|M|.$$

Combining the inequalities, we obtain

$$|M^*| \leq 2|M| \quad \Longrightarrow \quad |M| \geq \tfrac{1}{2}|M^*|.$$

Thus, the greedy algorithm produces a $\frac{1}{2}$-approximation. ⊛

Definition 3.5.1. Let G be a graph and let $M \subseteq E(G)$ be a matching. An M-alternating path is a path where every alter edge is in M. An M-augmenting path is an M-alternating path whose endpoints are unsaturated by M.

Theorem 3.5.1 (Berge, 1953). Let G be a graph and let $M \subseteq E(G)$ be a matching. Then M is a maximum matching if and only if there is no M-augmenting path.

Proof.

($\Rightarrow$) Suppose that M is a maximum matching. We show that there is no M-augmenting path.

Assume for contradiction that there exists an M-augmenting path P. By definition, P is an alternating path whose edges alternate between edges not in M and edges in M, and whose endpoints are *unsaturated* by M (i.e., not incident to any edge in M).

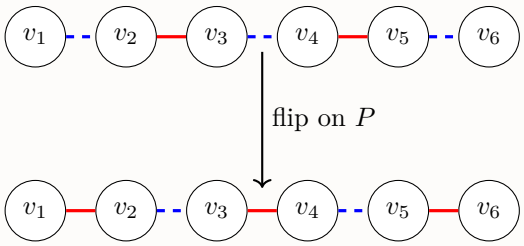
Define

$$M' := M \,\Delta\, E(P).$$

Since P is alternating, this operation replaces the edges of M on P with the edges not in M. Because both endpoints of P are unsaturated by M, the path contains one more edge not in M than in M. Hence,

$$|M'| = |M| + 1.$$

Before flip on P



After flip: $M' = M \Delta E(P)$

$= E(P) \setminus M$ $= E(P) \cap M$

Moreover, M' is still a matching, since along the path we never create two edges sharing a common endpoint.

This contradicts the maximality of M. Therefore, no M-augmenting path exists.

($\Leftarrow$) Suppose that there is no M-augmenting path. We show that M is a maximum matching.

Let M^* be a maximum matching in G, and consider

$$H := M \,\Delta\, M^*.$$

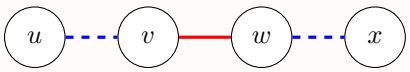
Each vertex in H has degree at most 2, since both M and M^* are matchings. Thus every connected component of H is either an even cycle or a path. Moreover, each component is alternating between edges in M and edges in M^*.

Consider a path component P. Along P, the edges alternate between M and M^*, so the numbers of edges from the two matchings differ by at most one.

If $|M^*| > |M|$, then there must exist a component P in which the number of edges from M^* exceeds the number from M. Such a component must be a path whose first and last edges lie in M^*.

Therefore, the endpoints of P are not incident to any edge in M, i.e., they are *unsaturated* by M. Hence, P is an M-augmenting path.

one endpoint incident to M^*, the other to M (since $|M^*| > |M|$)



$= M^* \setminus M$

$= M \setminus M^*$

This contradicts the assumption that no such path exists. Therefore, $|M| = |M^*|$, and thus M is a maximum matching. ■

Lecture 10

31 March.

alternative proof of Hall's theorem. Let $G = (X \cup Y, E)$ be a bipartite graph, and let M be a maximum matching.

Suppose, for contradiction, that M does not saturate all vertices in X. Let

$$U = \{x \in X : x \text{ is unmatched in } M\}.$$

Starting from U, explore all vertices reachable via M-alternating paths:

- from X to Y: along edges not in M,

- from Y to X: along edges in M.

Let $S \subseteq X$ and $T \subseteq Y$ be the sets of vertices in X and Y, respectively, that are reachable from U with the above rule.

We claim that every vertex in T is matched by M, and its matched partner lies in S. Indeed, if some $y \in T$ were unmatched, then there would exist an M-augmenting path starting from some $u \in U$ and ending at y, contradicting the maximality of M.

Thus every $y \in T$ is matched to a vertex in S, so

$$|T| \leq |S|.$$

Next, we claim that $N(S) = T$. By construction, every neighbor of a vertex in S that is reached lies in T, so $N(S) \subseteq T$. Conversely, every vertex in T is reached from some vertex in S, so $T \subseteq N(S)$. Hence,

$$N(S) = T.$$

Finally, since $U \subseteq S$ and $U \neq \emptyset$, there is at least one vertex in S that is unmatched, while every vertex in $T = N(S)$ is matched to a distinct vertex in S. Therefore,

$$|N(S)| = |T| < |S|.$$

This violates Hall's condition. Hence, if Hall's condition holds, there exists a matching that saturates X.

■

3.6 The Augmenting Path Algorithm (APA)

Now we want to design an algorithm such that

- Input: A bipartite graph $(X \cup Y, E)$ and a matching M.
- Output: Either an M-augmenting path or a vertex cover $C \subseteq X \cup Y$ with $|C| = |M|$, where C is a minimum vertex cover.

Remark 3.6.1. If M is a maximum matching, then M-augmenting path does not exists and since $\alpha'(G) = \beta(G)$, so a vertex cover C with $|C| = |M|$ exists. If M is not maximum, then M-augmenting path exists and we will return one.

If we can show that such an algorithm exists, then we can design an algorithm to find maximum matching:

Algorithm 3.1: Maximum Matching Algorithm (MMA)

Data: Bipartite graph $G = (X \cup Y, E)$.
Result: Maximum matching $M \subseteq E$ and minimum vertex cover $C \subseteq X \cup Y$ s.t. $|C| = |M|$.

```
M ← ∅
Run the augmenting path algorithm on M. if we got an augmenting path then
    flip edges to get a larger matching M, and repeat.
else
    we have a maximum matching M and a minimum vertex cover C (the vertex cover can be
     obtained from the augmenting path algorithm).
```

Observation. Since $|M|$ is strictly increasing, we only run the augmenting path algorithm at most $|X|$ times. If APA is polynomial, so is MMA.

The idea of APA is to start from unsaturated vertices of X, which are all in a set U, and see which vertices we can reach with M-alternating paths. If we can reach an unsaturated vertex in Y, then there is an M-augmenting path, and we can return it. If we have explored all M-alternating path starting from U, and finds that there is not any augmenting path, then M is a maximum matching, and we can

return the corresponding minimum vertex cover. We can maintain 3 sets of vertices:

$$S = \{\text{vertices we can reach in } X \text{ via } M\text{-alternating path from } U\}$$
$$T = \{\text{vertices in } Y \text{ we can reach via } M\text{-alternating paths from } U\}$$
$$Q = \{\text{vertices in } X \text{ where neighborhoods we have explolred}\}$$

Algorithm 3.2: Augmenting Path Algorithm

Data: A bipartite graph $(X \cup Y, E)$ and a matching M.
Result: Either an M-augmenting path or a vertex cover $C \subseteq X \cup Y$ with $|C| = |M|$.

```
1  S ← U, T ← ∅, Q ← ∅
2  while True do
3      if S = Q then
4          M is a maximum matching, and we return a vertex cover T ∪ (X \ S).
5      else
6          Let x ∈ S \ Q be arbitrary.
7          if x has any neighborhood y ∈ Y that is unsaturated then
8              STOP: M-augmenting path found: M-alternating path to x, together with {x, y}.
9          else
10             for every neighborhood y of x that is not in T do
11                 T ← T ∪ {y}.
12                 add the M-neighborhood x' of y to S.
13         Q ← Q ∪ {x}.
```

Observation. Since any vertex in U is unsaturated by M, any M-alternating path starting from U has the first endpoint (starting point) unsaturated by M, and thus once we find an $y \in T$ is unsaturated by M, then there must be an M-augmenting path.

Remark 3.6.2. From line 9, it means all neighbors of x are saturated by M, suppose y is a neighbor of x saturated by M. Then

- Case 1: x is the M-partner of y. But we have reached x, so we must went through this edge from y to x. But we can explore x here since $x \in S \setminus Q$, so x has been added to S when we explore $\{y, x\}$. Note that after this round x is added to Q, so we will not recompute x.
- Case 2: x is not the M-partner of y. Then we can walk from x through $\{x, y\}$ to y, which can extend the alternating path.

Observation. Each iteration increases the size of Q by 1, so at most $|X|$ iterations. Also, every edge is considered at most once, so the running time is $O(|E|)$.

3.6.1 Correction of APA

If an augmenting path is returned, each vertex we reached, we kept track of how we reached it, so we do get a path, where the path traces back to U, and thus endpoints are unsaturated, which means this path is an augmenting path (Note that this path is altering since edges from X to Y in this path not in the matching and edges from Y to X are in the matching).

If no path is found, we return a cover $C = T \cup (X \setminus S)$.

(1) C is a cover: Since $S = Q$, we have explored all neighbors of S, and all neighbors of S should be in T, so there are no edges between S and $Y \setminus T$, i.e. for each edge $e \in E(G)$, one of endpoint of e must be in $T \cup (X \setminus S)$, i.e. $T \cup (X \setminus S)$ covers $E(G)$.

(2) $|C| = |M|$. Every vertex in T is saturated, with its M-partner in S, otherwise there is an augmenting path. Every vertex in $X \setminus S$ is saturated with its M-partner outside T. (Every vertex

in $X \setminus S$ saturated since S is initially U and keep getting larger, so $X \setminus S$ only contains vertices that is saturated by M, and if x's M-partner is in T, then $x \in S$, which is a contradiction. Also, remember that no edges bewteen S and $Y \setminus T$, so $|C|$ counts all edges in M.)

Remark 3.6.3. In fact, C is a minimum vertex cover since we return C when M is a maximum matching and $|C| = |M|$, and in bipartite graph $\alpha'(G) = \beta(G)$, so C is a minimum vertex cover.

3.7 Maximum weight matchings

- Setting: We have a weighting on the edges of $K_{n,n}$: $w : E(K_{n,n}) \to \mathbb{R}$.
- Goal: Find a perfect matching M that maximizes $w(M) = \sum_{e \in M} w(e)$.

Maximum weight matching problem can help us solve to maximum matching problem: Given a bipartite $G \subseteq K_{n,n}$, then we define

$$w(e) = \begin{cases} 1, & \text{if } e \in E(G); \\ 0, & \text{if } e \notin E(G). \end{cases}$$

Then maximum weight of a perfect matching is the max size of a matching in G.

Definition 3.7.1 (Duality). Let w be a weighting of $E(K_{n,n})$. Then $c : V(K_{n,n}) \to \mathbb{R}$ is a (weighted) cover of w if

$$\forall \{x, y\} \in E(K_{n,n}), \quad w(\{x, y\}) \leq c(x) + c(y).$$

The value of the cover is

$$\text{val}(c) = \sum_{v \in V(K_{n,n})} c(v).$$

Proposition 3.7.1. Let w be a weighting of $E(K_{n,n})$, let M be a perfect matching, and let c be a weighted cover. Then, $w(M) \leq \text{val}(c)$, and we have equality iff

$$w(\{x, y\}) = c(x) + c(y) \text{ for all } \{x, y\} \in M.$$

Proof.

$$w(M) = \sum_{\{x,y\} \in M} w(\{x, y\}) \leq \sum_{\{x,y\} \in M} (c(x) + c(y)) = \sum_{v \in V(K_{n,n})} c(v) = \text{val}(c)$$

since c is a cover and M is a perfect matching. Also, the equality holds iff

$$w(\{x, y\}) = c(x) + c(y) \text{ for all } \{x, y\} \in M.$$

■

Theorem 3.7.1 (Duality for max weight matching). For any weighting of $E(K_{n,n})$,

$$\max \{w(M) : M \text{ is a perfect matching}\} = \min \{\text{val}(c) : c \text{ is a weighted cover}\}.$$

Proof. We prove this algorithmically:

Algorithm 3.3: Hungarian Algorithm (Maximum Weight Perfect Matching)

Input: A complete bipartite graph $K_{n,n} = (X \cup Y, E)$ with weight function $w : E \to \mathbb{R}$.

Output: A perfect matching $M \subseteq E$ of maximum total weight, together with a weighted cover $c = (u, v)$ certifying optimality.

Initialization: Choose an initial weighted cover $c = (u, v)$, where $u : X \to \mathbb{R}$, $v : Y \to \mathbb{R}$, satisfying $u(x) + v(y) \geq w(x, y), \quad \forall (x, y) \in E$. For example, we can choose $u(x) = \max_{y \in Y} w(x, y)\ \forall x \in X$ and $v(y) = 0\ \forall y \in Y$.

while *true* **do**

  Construct the equality graph: $G_{u,v} = \{(x, y) \in E \mid u(x) + v(y) = w(x, y)\}$.

  Use the MMA to find a maximum matching M in $G_{u,v}$.

  if *M is a perfect matching* **then**

    return M together with $c = (u, v)$ as a certificate of optimality.

  Let Q be the minimum vertex cover of $G_{u,v}$ obtained from the MMA.

  Define: $R = Q \cap X, \quad T = Q \cap Y$.

  Let

$$\varepsilon = \min_{\substack{x \notin R \\ y \notin T}} \big(u(x) + v(y) - w(x, y)\big).$$

  Update the weighted cover $c = (u, v)$ as follows:

$$u(x) \leftarrow \begin{cases} u(x) - \varepsilon, & x \notin R \\ u(x), & x \in R \end{cases} \qquad v(y) \leftarrow \begin{cases} v(y) + \varepsilon, & y \in T \\ v(y), & y \notin T \end{cases}$$

  This preserves the weighted cover condition and enlarges $G_{u,v}$.

Since for every perfect matching M and weighted cover c in $K_{n,n}$, we have shown that $w(M) \leq \mathrm{val}(c)$, so

$$\max \{w(M) : M \text{ is a perfect matching}\} \leq \min \{\mathrm{val}(c) : c \text{ is a weighted cover}\},$$

and this algorithm shows finally we can find a perfect matching M and a weighted cover c s.t. $w(M) = \mathrm{val}(c)$. Thus, the equality holds. ■

Lecture 11

Now we explain why Hungarian Algorithm is correct. 7 April.

Remark 3.7.1. Observe that $\varepsilon > 0$, since if $x \notin R$ and $y \notin T$, then $\{x, y\}$ is not covered by Q. Hence, $\{x, y\} \notin E(G_{u,v})$. This shows $w(\{x, y\}) < u(x) + v(y)$.

Proposition 3.7.2. After the update step, the pair $c = (u, v)$ remains a weighted cover, i.e.,

$$u(x) + v(y) \geq w(x, y), \quad \forall (x, y) \in E.$$

Proof. Let u', v' denote the updated functions. We verify that

$$u'(x) + v'(y) \geq w(x, y), \quad \forall (x, y) \in E,$$

by considering all possible cases.

Case 1: $x \in R,\ y \notin T$.
In this case, no update is applied:

$$u'(x) = u(x), \quad v'(y) = v(y).$$

Hence,

$$u'(x) + v'(y) = u(x) + v(y) \geq w(x, y),$$

since $c = (u, v)$ is a weighted cover.

Case 2: $x \in R,\ y \in T$.
We have:

$$u'(x) = u(x), \quad v'(y) = v(y) + \varepsilon.$$

Thus,

$$u'(x) + v'(y) = u(x) + v(y) + \varepsilon \geq w(x, y),$$

again using the feasibility of c.

Case 3: $x \notin R,\ y \in T$.
Here,

$$u'(x) = u(x) - \varepsilon, \quad v'(y) = v(y) + \varepsilon,$$

so

$$u'(x) + v'(y) = u(x) + v(y) \geq w(x, y).$$

Case 4: $x \notin R,\ y \notin T$.
In this case,

$$u'(x) = u(x) - \varepsilon, \quad v'(y) = v(y),$$

and hence

$$u'(x) + v'(y) = u(x) + v(y) - \varepsilon.$$

By the definition of ε,

$$\varepsilon = \min_{\substack{x \notin R \\ y \notin T}} \big(u(x) + v(y) - w(x, y)\big),$$

it follows that for all $x \notin R, y \notin T$,

$$\varepsilon \leq u(x) + v(y) - w(x, y).$$

Rearranging gives

$$u(x) + v(y) - \varepsilon \geq w(x, y),$$

and therefore

$$u'(x) + v'(y) \geq w(x, y).$$

Since all cases satisfy the inequality, we conclude that $c' = (u', v')$ is still a weighted cover. ■

Remark 3.7.2. When we update the weighted cover from (u, v) to (u', v'), the current matching M remains a valid matching in $G_{u',v'}$. To show this, we have to show for every $\{x, y\} \in M$, $\{x, y\} \in G_{u',v'}$, then since all edges in M are in $G_{u',v'}$, and vertices in $G_{u',v'}$ is same as $G_{u,v}$, so M is still a matching in $G_{u',v'}$. Now we show every $\{x, y\} \in M$ is in $G_{u',v'}$. Note that M is a maximum matching of $G_{u,v}$. Thus, $\{x, y\}$ satisfies $w(x, y) = u(x) + v(y)$. Now since Q is a minimum vertex cover of $G_{u,v}$, so $\{x, y\}$ is covered by Q, i.e. x or y is in Q. Note that exactly one of x, y is in Q, since $|Q| = |M|$ (since $G_{u,v}$ is bipartite) and Q covers M. Thus, either $x \in R, y \notin T$ or $x \notin R, y \in T$ will occur, and each case we have $u(x) + v(y) = u'(x) + v'(y)$, so we're done.

Consequently, the size of a maximum matching in the equality graph does not decrease:

$$\alpha'(G_{u',v'}) \geq \alpha'(G_{u,v}).$$

Let us consider the set of M-unsaturated vertices in X, i.e., vertices not covered by M. After the update, for each M-unsaturated vertex $x \in X$, we explore the M-alternating paths in $G_{u',v'}$. The update rule for (u', v') (Case 4 above, since some edges $\{x', y'\}$ has $\varepsilon = u(x') + v(y') - w(x', y')$) ensures that at least one new edge in $G_{u',v'}$ connects some vertex $x' \notin R$ to a vertex $y' \notin T$, which was previously not in the equality graph. Now since this new edge $\{x', y'\}$ has $x' \notin R$ and $y' \notin T$,

and since $R = Q \cap X$, where $Q = T' \cup (X \setminus S')$, where

$$\begin{aligned} U &= \{\text{vertices in } X \text{ that are unsaturated by } M\} \\ S' &= \{\text{vertices in } X \text{ that can be reached through } M\text{-alternating from } U\} \\ T' &= \{\text{vertices in } Y \text{ that can be reached through } M\text{-alternating from } U\}, \end{aligned}$$

so if $x \notin R = Q \cap X = X \setminus S'$, then x can be reached from U through M-altenating path. However, $y \notin T = Q \cap Y = T'$, which means y cannot be reached from U through M-alternating path. This means the update creates a new vertex in Y that can be reached by an M-alternating path from x, so the alternating tree enlarges.

Now we may think the edges in Case 2 disappear in $G_{u',v'}$ after the update, will it affect? In fact, if $\{x_1, y_1\}$ is a Case 2 edge appears in $G_{u,v}$, then $x_1 \in R$ and $y_1 \in T$. This shows x_1 cannot be reached from U through M-alternating path and y_1 can. However, $\{x_1, y_1\} \notin M$ since we have shown that for any edge in M, either $x_1 \in R$ or $y_1 \in T$. Hence, under this condition, $\{x_1, y_1\}$ is useless for enlarging alternating tree.

Since there are at most n vertices in Y, after at most n iterations of expanding the alternating tree from an M-unsaturated vertex in X, we either reach an M-unsaturated vertex in Y or we add a new reachable vertex to the equality graph. In the former case, an M-augmenting path exists, allowing us to increase the size of the matching by flipping edges of this augmenting paths:

$$|M| \longrightarrow |M| + 1.$$

Because the maximum size of a matching in a bipartite graph with n vertices on each side is n, this argument implies that within at most n^2 iterations, we must obtain a perfect matching. Thus, the algorithm terminates with a perfect matching, and the process is guaranteed to converge.

3.8 Stable Matching Problem

Suppose we have a complete bipartite graph $G = K_{n,n} = (X \cup Y, E)$, then we can define preference lists:

Preference lists Each vertex ranks the vertices on the other side in order of preference, i.e. $\forall x \in X$, $\exists \pi^{(x)} \in S_Y$ and $\forall y \in Y$, $\exists \pi^{(y)} \in S_X$.

Definition 3.8.1. Given a matching M, a pair $\{x, y\} \notin M$ is an unstable pair if x prefers y to their M-partner and y prefer x to their M-partner. A perfect matching M is a stable matching if there are no unstable pairs.

Theorem 3.8.1 (Gale-Shapley, 1962)**.** A stable matching always exists.

3.8.1 Gale-Shapley Proposal Algorithm

Every night, every man proposes to their top-ranked woman who haven't reject them. If every women receives exactly one proposal, then stop, and there is a stable matching formed by the woman and the man who proposed to her. Otherwise, every woman says "maybe" to her top suitor and rejects any other.

Remark 3.8.1. In each round, if a woman says "maybe" to a man, then this man can propose again to this woman in next round, but the men who are rejected by the woman cannot.

Remark 3.8.2. If the man receive "maybe" from a woman, then he will propose to this woman in the following rounds until he is rejected or the algorithm ends.

Claim 3.8.1. If the algorithm produces a matching, then it is stable.

Proof. Suppose not, suppose man x is matched with woman y and man x' is matched with woman y', and $\{x, y'\}$ is an unstable pair. Note that in the algorithm, the men move down their preference lists, the women move up their preference lists, so x proposed to y' before proposing to y, and y' only rejected x because she had a better option, so she prefer x' to x (If y' prefer x more than x', then when y' rejects x, there will always be a better option than x in the following rounds, but then when x' propose to y', x' will not be the best option and thus x' will be rejected by y'), so $\{x, y'\}$ is not a unstable pair, which is a contradiction. ⊛

Claim 3.8.2. The algorithm terminates.

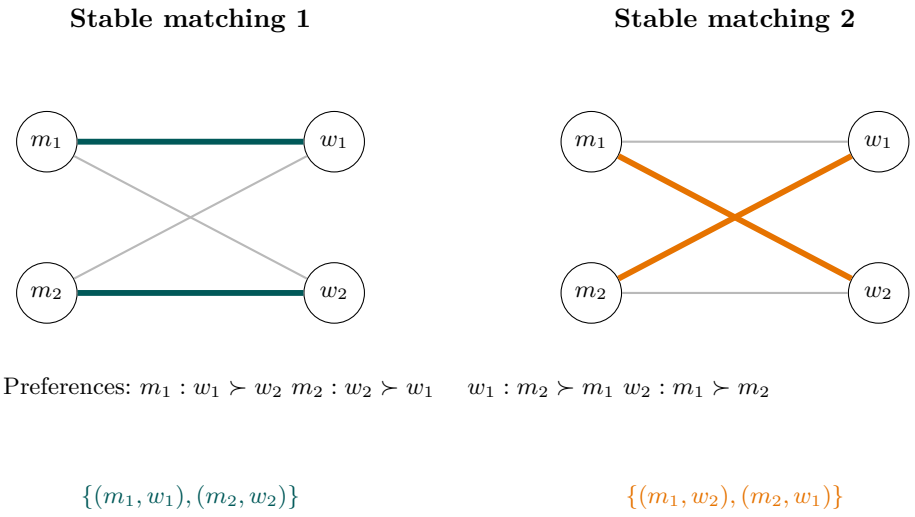
Proof. Suppose in an iteration, the i-th man is proposing to the j_i-th woman on their list (initially $j_i = 1$ for all i). Then, $\sum_{i=1}^{n} j_i$ is strictly increasing with every iteration. (If every man propose to different woman, then the algorithm ends. If two men propose to same woman, then one of them will be rejected and thus $\sum_{i=1}^{n} j_i$ increases in next round) Thus, it terminates within n^2 iterations. ⊛

Claim 3.8.3. The algorithm always produces a matching.

Proof. If a man has been rejected by everyone, then it implies every woman has been proposed to, and thus every woman has n distinct "maybe" partner, but there are only n men, so it is impossible. ⊛

Lecture 12

Remark 3.8.3. The stable matching in $K_{n,n}$ may not be unique. 10 April.



Preferences: $m_1 : w_1 \succ w_2$ $m_2 : w_2 \succ w_1$ $w_1 : m_2 \succ m_1$ $w_2 : m_1 \succ m_2$

$\{(m_1, w_1), (m_2, w_2)\}$ $\{(m_1, w_2), (m_2, w_1)\}$

The graph and preference lists are the same in both panels, but the highlighted edges form two different stable matchings. This illustrates that stable matchings need not be unique.

Chapter 4

Connectivity

4.1 Vertex Connectivity

Recall that a graph G is connected if there is a path between any two vertices.

Definition 4.1.1. Let $G = (V, E)$ be a graph. A set $S \subseteq V$ is a separating set if $G - S$ is disconnected.

Definition 4.1.2. The connectivity of a graph, denoted $\kappa(G)$, is the minimum size of a set $S \subseteq V$ such that either S is a separating set or $G - S$ is a single vertex (only for $G = K_n$ for the latter case).

Definition 4.1.3. We say a graph G is k-connected if $\kappa(G) \geq k$.

Observation. For any G, $\kappa(G) \leq \delta(G)$, where $\delta(G)$ is the minimum degree of G.

Proof. For any vertex v, $S = N(v)$ either disconnects v from the rest of the graph, or leaves v as the remaining vertex. ■

Example 4.1.1. $\kappa(K_{n,m}) = \delta(K_{n,m}) = \min\{n, m\}$.

Question. How many edges does an n-vertex k-connected graph have to have?

Answer. For $k = 2$, the answer is n, since we can consider C_n, and if it has only $n - 1$ edges, then since it is connected, so it is a tree, but then a tree is 1-connected.

For $k \geq 2$, the answer is $\lceil \frac{kn}{2} \rceil$. For the lower bound, since $\kappa(G) \geq k$, so $\delta(G) \geq k$. Hence,

$$e(G) = \frac{1}{2} \sum_v \deg(v) \geq \frac{n\delta(G)}{2} \geq \frac{nk}{2}.$$

For the upper bound, . ⊛ DIY

Theorem 4.1.1 (Chvatal-Erdos, 1972). If $G \neq K_2$ and $\kappa(G) \geq \alpha(G)$, where $\alpha(G)$ is the size of largest independent set, then G is Hamiltonain.

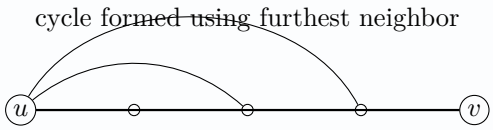
Proof. Let C be a largest cycle in G, and let $\kappa(G) = k$.

Claim 4.1.1. $|C| \geq k + 1$.

Proof. Let P be a maximal path in G with endpoint u. Since P is maximal, every neighbor of u lies on P, so $N(u) \subseteq V(P)$. Hence

$$|N(u)| \geq \delta(G) \geq \kappa(G) = k.$$

Let v be the neighbor of u furthest along P. Then the subpath of P from u to v, together with the edge uv, forms a cycle. Because there are at least k neighbors of u on P, this cycle has at least $k+1$ vertices.



P (maximal path)

⊛

If C is Hamilton, then we are done. So assume C is not Hamilton. Let Q be a component of $G - C$. Write $V(C) = \{v_1, \ldots, v_t\}$ in cycle order.

Claim 4.1.2. At least k vertices v_i have a neighbor in Q.

Proof. Let $S \subseteq V(C)$ be the set of vertices of C that have a neighbor in Q. If every vertex of C lies in S, then $|S| = |C| \geq k+1$, and in particular $|S| \geq k$. Otherwise, S separates the component Q from the vertices $V(C) \setminus S$, because every path from Q to a vertex of C with no neighbor in Q must pass through a vertex of S. Thus S is a vertex cut, so

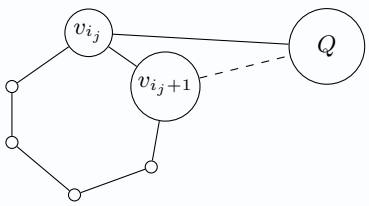
$$|S| \geq \kappa(G) = k.$$

Therefore at least k vertices of C have a neighbor in Q. ⊛

Let $v_{i_1}, v_{i_2}, \ldots, v_{i_k}$ be vertices of C that have a neighbor in Q.

Claim 4.1.3. v_{i_j+1} has no neighbor in Q for all $1 \leq j \leq k$.

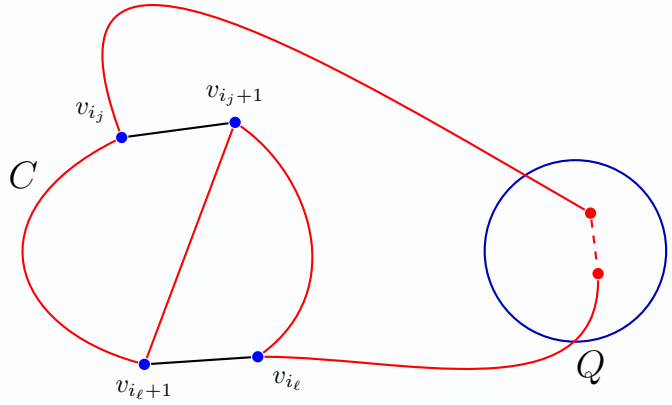
Proof. The figure indicates the key idea: if both v_{i_j} and v_{i_j+1} had neighbors in Q, then one could leave C through v_{i_j}, travel through Q, and return to C at v_{i_j+1}, producing a cycle longer than C.



⊛

Claim 4.1.4. $\{v_{i_j+1} : 1 \le j \le k\}$ is an independent set.

Proof. Assume two vertices v_{i_j+1} and $v_{i_\ell+1}$ are adjacent. Since v_{i_j} and v_{i_ℓ} both have neighbors in the same component Q, there is a path in Q joining them. Using that path, together with the assumed edge $v_{i_j+1}v_{i_\ell+1}$ and the appropriate arcs of C, we obtain a cycle that is longer than C. This contradiction shows that no two vertices in $\{v_{i_j+1} : 1 \le j \le k\}$ are adjacent.



⊛

Now choose any vertex of Q. Since each v_{i_j+1} has no neighbor in Q, that vertex is adjacent to none of the vertices in

$$\{v_{i_j+1} : 1 \le j \le k\}.$$

Hence this set, together with any vertex of Q, is an independent set of size $k+1$. Thus $\alpha(G) \ge k+1 > k = \kappa(G)$, contradicting the assumption $\kappa(G) \ge \alpha(G)$.

Therefore C must be Hamiltonian, and so G is Hamiltonian. ■

4.2 Edge Connectivity

Lecture 13

21 April.

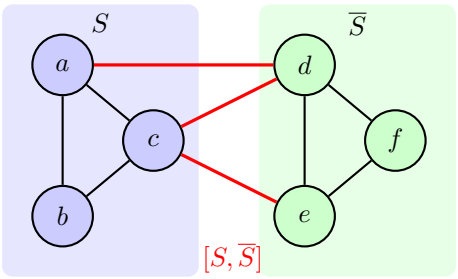
Definition 4.2.1 (Edge connectivity)**.** The edge-connectivity of a graph G is

$$\kappa'(G) := \min\{|F| : F \subseteq E(G), G - F \text{ is disconnected}\}.$$

Definition 4.2.2. We say G is k-edge connected if $\kappa'(G) \ge k$.

Definition 4.2.3. Given a graph $G = (V, E)$, an edge cut $[S, \overline{S}]$, where $S \subseteq V$, $S \neq \varnothing, V$, and $\overline{S} = V \setminus S$, is

$$[S, \overline{S}] = \{\{x, y\} \in E : x \in S, y \in \overline{S}\}.$$



Observation. Any minimal disconnecting set of edges will be edge cut, and any edge cut is a disconnecting set of edges. Therefore,

$$\kappa'(G) = \min\left\{\left|\left[S,\overline{S}\right]\right| : \varnothing \neq S \subsetneq V\right\}.$$

Theorem 4.2.1 (Whitney, 1932). For any graph G, $\kappa(G) \leq \kappa'(G) \leq \delta(G)$.

Proof. We first show $\kappa'(G) \leq \delta(G)$: Let v be a vertex of minimum degree, then removing all its edges isolates it, i.e.

$$\left|\left[\{v\}, V \setminus \{v\}\right]\right| = \deg(v).$$

As for $\kappa(G) \leq \kappa'(G)$. Let $F \subseteq E(G)$ where $|F| = \kappa'(G)$ s.t. $G - F$ is disconnected. Let $F = [S,\overline{S}]$. If $\exists x \in S$ and $y \in \overline{S}$ s.t. $\{x,y\} \notin E(G)$, then for every $\{u,v\} \in [S,\overline{S}]$, if $u \neq x$, remove u, otherwise remove v. This ensures x, y are in the remaining graph. Hence, we can remove $\leq |F|$ vertices to make remove all edges in F, and at the end, x, y are disconnected. Hence, $\kappa(G) \leq |F| = \kappa'(G)$. Otherwise, if $\{x,y\} \in E(G)$ for all $x \in S$ and $y \in \overline{S}$, then

$$|F| = |S| \cdot |\overline{S}| = |S| \cdot (n - |S|) \geq n - 1.$$

Hence, $\kappa(G) \leq n - 1 \leq |F| = \kappa'(G)$. ■

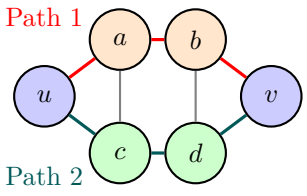
Exercise 4.2.1 (HW). $\forall k \leq \ell \leq m$, $\exists$ a graph G with

$$\kappa(G) = k, \quad \kappa'(G) = \ell, \quad \delta(G) = m.$$

Question. "Is G k-connected?" is a decision problem of P or NP or co-NP?

Answer. It is co-NP since we can verify a disconnected set of size $< k$ in polynomial time. Also, if k is a constant, then we can check if all $\binom{n}{k-1}$ vertex subsets are not disconnecting sets in polynomial time by brute forces. ⊛

Characterization of k-connected graphs If $k = 2$, then it means we can remove any vertex from the graph and the graph remains connected. Hence, we have a sufficient condition for 2-connectivity: For any two vertices u, v, there are two internally vertex disjoint uv-path, i.e. two uv-path with only u, v are common vertices of 2 paths.

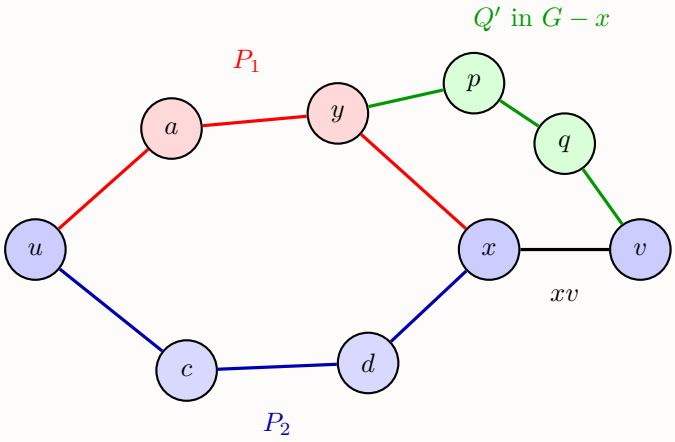


Theorem 4.2.2 (Whitney, 1932)**.** A graph is 2-connected if and only if for every $u, v \in V(G)$, there are two internally disjoint uv-paths.

Proof.

($\Leftarrow$) Consider $G - x$ for any $x \in V$, then for any $u, v \neq x$, we have two internally disjoint uv-path, where x is in at most one if them. Hence, the path without x is in $G - x$, which means $G - x$ is connected, i.e. G is 2-connected.

($\Rightarrow$) We do induction on $d_G(u, v)$. For the base case: $d_G(u, v) = 1$, then since 2-connected implies 2-edge-connected by Theorem 4.2.1, so $G - \{u, v\}$ is connected. Hence, there is a uv-path P in $G - \{u, v\}$. Thus, $\{u, v\}$ and P are two internally disjoint uv-path. Now we do the induction step: Let $d_G(u, v) \geq 2$. Let Q be a shortest uv-path. Let x be the vertex preceding v on Q, then $d_G(u, x) = d_G(u, v) - 1$. By induction hypothesis, there are two internally-disjoint ux-paths in G, say P_1, P_2. Since G is 2-connected, $G - \{x\}$ is connected, so there is a uv-path in Q' in $G - \{x\}$. Starting from v, trace back along Q' until the first meet with $P_1 \cup P_2$, switch to that path and go back to u. Using the other path, go from u to x, and use the edge from x to v. Hence, we have two internally disjoint uv-paths.



One uv-path: $uP_1y \cup yQ'v$
Other uv-path: $uP_2x \cup xv$

■

Theorem 4.2.3. A graph G is 2-connected if and only if $\delta(G) \geq 1$ and for any two edges in the graph, there is a cycle combining both.

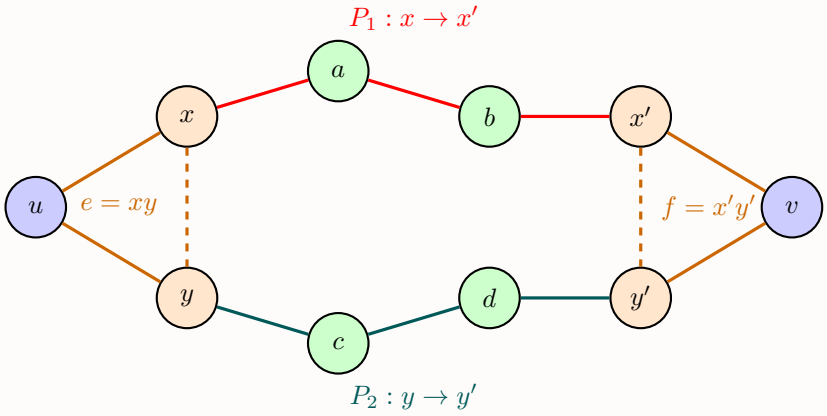
Proof. We first give a lemma:

> **Lemma 4.2.1** (Extension Lemma)**.** If G is k-connected, and G' is obtained by adding a vertex v of degree $\geq k$ to G, then G' is k-connected.
>
> **Proof.** Let $S \subseteq V(G')$ and $|S| \leq k - 1$, then we need to show that $G' - S$ is connected. If $v \in S$, then $G' - S = G - S$, which is connected since G is k-connected. If $S \subseteq V(G)$, then $G - S$ is connected since G is k-connected. Since v has $\geq k$ neighbors, and $|S| \leq k - 1$, so v still has a neighbor in $G - S$, which shows $G' - S$ is connected. ■

Now we prove the theorem.

($\Leftarrow$) Given $u, v \in V(G)$, let e, f be edges containing u, v, respectively. Since e, f are on a cycle containing u, v, so there are two internally disjoint uv-path, i.e. G is 2-connected.

($\Rightarrow$) Let G be 2-connected, and $e, f \in E(G)$. Let G' be obtained by adding a vertex u to the endpoints of e to G. Then by the extension lemma, G' is 2-connected. Now let G'' be obtained by adding a vertex v adjacent to endpoints of f to G', then G'' is 2-connected by extension lemma. By Whitney, there exists internally disjoint paths P_1, P_2 from u to v. Consider $P_1 \cup P_2$, replacing $x - u - y$ by $\{x, y\} = e$ and $x' - v - y'$ with $\{x', y'\} = f$, we get a cycle in G containing e and f.



■

Corollary 4.2.1. 2-connectivity problem is in NP ∩ co-NP.

Theorem 4.2.4 (Global vertex Menger). G is k-connected if and only if for any $u, v \in V(G)$, $\exists$ k internally-disjoint uv-path.

Proof.

($\Leftarrow$) Obvious.

($\Rightarrow$) Harder.

Remark 4.2.1. This shows k-connectivity is in NP ∩ co-NP.

Remark 4.2.2. However, it is really hard to prove the "harder" direction directly. Instead, we use Local vertex Menger to prove this theorem.

■

Definition 4.2.4. Given $G = (V, E)$, $u, v \in V$. Then,

$$\kappa(u, v) = \min\{|S| : S \subseteq V \setminus \{u, v\} \text{ and } \nexists uv\text{-path in } G - S \ (S \text{ is a } uv\text{-separator})\}$$
$$\lambda(u, v) = \max\{|P| : P \text{ is a collection of internally disjoint } uv\text{-path}\}.$$

Theorem 4.2.5 (Local vertex Menger). If $G = (V, E)$ is a graph, and $u, v \in V$ with $\{u, v\} \notin E$, then $\kappa(u, v) = \lambda(u, v)$.

Proof. Suppose $\kappa(u, v) = k$ and $\lambda(u, v) = \lambda$, then note that $k \geq \lambda$ since we have to remove at least one vertex from each internally disjoint uv-path to disconnect u and v. Now suppose we have λ internally disjoint uv-paths, say $P_1, \ldots, P_\lambda$, then any other uv-path must intersect with one of P_i at some edge. Now suppose the previous vertex of v in P_i is w_i (note that $w_i \neq u$ since $\{u, v\} \notin E$), then note that in $G - \{w_1, \ldots, w_\lambda\}$ u and v are disconnected. Thus, $k \leq \lambda$, and we're done. ■

Claim 4.2.1. Local vertex Menger implies Global vertex Menger.

Proof. Let G be k-connected, $u, v \in V(G)$. If $\{u, v\} \notin E(G)$, then Local vertex Menger implies $\lambda(u, v) = \kappa(u, v) \geq \kappa(G) \geq k$. Hence, we have k internally disjoint uv-paths, which is what global vertex Menger states.

If $\{u, v\} \in E(G)$: Let $G' \equiv G - \{u, v\}$, so $\kappa(G') \geq \kappa(G) - 1$. Apply local vertex Menger to G':

$$\lambda_{G'}(u, v) = \kappa_{G'}(u, v) \geq \kappa(G') \geq \kappa(G) - 1 \geq k - 1.$$

Hence, in G', there are $k - 1$ internally disjoint uv-paths, and in G we have these $k - 1$ paths and $\{u, v\}$, so we have k internally disjoint uv-paths in G, and we're done. ⊛

Lecture 14

Edge Menger

24 April.

Definition 4.2.5.

$$\kappa'(u, v) = \min\{|F| : F \subseteq E(G), u, v \text{ disconnected in } G - F\}$$
$$\lambda'(u, v) = \max\{|P| : P \text{ is a set of pairwise edge-disjoint } uv\text{-path}\}$$

Theorem 4.2.6 (Local edge Menger Theorem). $\forall u, v \in V(G)$, $\kappa'(u, v) = \lambda'(u, v)$.

Proof. Follows from local vertex Menger. ■ HW

Theorem 4.2.7 (Global edge Menger Theorem). If G is k-edge-connected, then for any $u, v \in V(G)$, there are k pairwise edge-disjoint uv-paths.

Chapter 5

Network Flows

Definition 5.0.1. A network is a 4-tuple $(\vec{D}, s, t, c)$, where

- $\vec{D} = (V, E)$ is a directed multigraph s.t. V is a set of vertices and $E \subseteq V^2$ is a multiset of directed edges: $e = (e^-, e^+)$, where e^- is the endpoint of e, and e^+ is the starting point of e.
- $s \in V$ is a source vertex.
- $t \in V$ is a sink vertex.
- $c : \vec{E} \to \mathbb{R}_{\geq 0}$ are the capacities of the edges.

Definition 5.0.2. A flow is a function $f : \vec{E} \to \mathbb{R}_{\geq 0}$. A flow is feasible if

- (Capacity constraint) $0 \leq f(\vec{e}) \leq c(\vec{e})$ for all $\vec{e} \in E(G)$.
- (Conservation of flow) $\forall v \neq s, t$,

$$\sum_{\vec{e}: v = e^-} f(\vec{e}) = \sum_{\vec{e}: v = e^+} f(\vec{e}).$$

The value of the flow is

$$\mathrm{val}(f) = \sum_{\vec{e}: t = e^+} f(\vec{e}) - \sum_{\vec{e}: t = e^-} f(\vec{e}).$$

Our objective is to maximize the value of a feasible flow.

Definition 5.0.3. A source-sink cut is

$$\left[S, \overline{S}\right] = \left\{\vec{e} = (x, y) \in \vec{E} : x \in S, y \in \overline{S}\right\},$$

where $S \subseteq V, \overline{S} \subseteq V \setminus S, s \in S, t \in \overline{S}$. The capacity of the cut is

$$\mathrm{cap}(S, \overline{S}) = \sum_{\vec{e} \in [S, \overline{S}]} c(\vec{e}).$$

Lemma 5.0.1 (Weak Duality)**.** For any feasible flow f, and any source-sink cut $[S, \overline{S}]$,

$$\mathrm{val}(f) \leq \mathrm{cap}(S, \overline{S}).$$

Proof.

$$\mathrm{cap}(S, \overline{S}) = \sum_{\vec{e} \in [S, \overline{S}]} c(\vec{e}) \geq \sum_{\vec{e} \in [S, \overline{S}]} f(\vec{e}) \geq \sum_{\vec{e} \in [S, \overline{S}]} f(\vec{e}) - \sum_{\vec{e} \in [\overline{S}, S]} f(\vec{e}) = \mathrm{val}(f),$$

where the last equality is from Lemma 5.0.2. ■

Lemma 5.0.2 (Conservation). For any source-sink cut $[Q, \overline{Q}]$ and a feasible flow f,

$$\sum_{\vec{e} \in [Q,\overline{Q}]} f(\vec{e}) - \sum_{\vec{e} \in [\overline{Q},Q]} f(\vec{e}) = \text{val}(f).$$

Proof. We can do induction on $|\overline{Q}|$.

- Base case: $|\overline{Q}| = 1$, then $\overline{Q} = \{t\}$. Hence,

$$\sum_{\vec{e} \in [Q,\overline{Q}]} f(\vec{e}) - \sum_{\vec{e} \in [\overline{Q},Q]} f(\vec{e}) = \sum_{\vec{e}: e^+ = t} f(\vec{e}) - \sum_{\vec{e}: e^- = t} f(\vec{e}) = \text{val}(f).$$

- Induction step: Now if $|\overline{Q}| \geq 2$. Let $x \in \overline{Q}$ where $x \neq t$. Consider $R = Q \cup \{x\}$, then $[R, \overline{R}]$ is a source-sink cut with $|\overline{R}| < |\overline{Q}|$. By induction,

$$\begin{aligned}
\text{val}(f) &= \sum_{\vec{e} \in [R,\overline{R}]} f(\vec{e}) - \sum_{\vec{e} \in [\overline{R},R]} f(\vec{e}) \\
&= \sum_{\substack{\vec{e} \in [R,\overline{R}] \\ x \neq e^-}} f(\vec{e}) + \sum_{\substack{\vec{e} \in [R,\overline{R}] \\ x = e^-}} f(\vec{e}) - \sum_{\substack{\vec{e} \in [\overline{R},R] \\ x = e^+}} f(\vec{e}) - \sum_{\substack{\vec{e} \in [\overline{R},R] \\ x \neq e^+}} f(\vec{e}) \\
&= \sum_{\substack{\vec{e} \in [Q,\overline{Q}] \\ x \neq e^+}} f(\vec{e}) - \sum_{\substack{\vec{e} \in [\overline{Q},Q] \\ x \neq e^-}} f(\vec{e}) + \sum_{\vec{e} \in [x,\overline{R}]} f(\vec{e}) - \sum_{\vec{e} \in [\overline{R},x]} f(\vec{e}) \\
&= \left(\sum_{\vec{e} \in [Q,\overline{Q}]} f(\vec{e}) - \sum_{\vec{e} \in [Q,x]} f(\vec{e}) \right) - \left(\sum_{\vec{e} \in [\overline{Q},Q]} f(\vec{e}) - \sum_{\vec{e} \in [x,Q]} f(\vec{e}) \right) + \sum_{\vec{e} \in [x,\overline{R}]} f(e) - \sum_{\vec{e} \in [\overline{R},x]} f(\vec{e}) \\
&= \sum_{\vec{e} \in [Q,\overline{Q}]} f(\vec{e}) - \sum_{\vec{e} \in [\overline{Q},Q]} f(\vec{e}) + \sum_{\vec{e} \in [x,Q \cup \overline{R}]} f(\vec{e}) - \sum_{\vec{e} \in [Q \cup \overline{R},x]} f(\vec{e}) \\
&= \sum_{\vec{e} \in [Q,\overline{Q}]} f(\vec{e}) - \sum_{\vec{e} \in [\overline{Q},Q]} f(\vec{e}) + \overbrace{\sum_{\vec{e}: x = e^-} f(\vec{e}) - \sum_{\vec{e}: x = e^+} f(\vec{e})}^{=0} \\
&= \sum_{\vec{e} \in [Q,\overline{Q}]} f(\vec{e}) - \sum_{\vec{e} \in [\overline{Q},Q]} f(\vec{e}).
\end{aligned}$$

■

Lecture 15

28 April.

Corollary 5.0.1. For any feasible flow f,

$$\text{val}(f) = \sum_{\vec{e}: e^- = s} f(\vec{e}) - \sum_{\vec{e}: e^+ = s} f(\vec{e}).$$

Proof. This can be immediately obtained from Lemma 5.0.2 by letting $Q = \{s\}$. ■

Remark 5.0.1. We have shown that the value of a flow is how much is left at the sink, which is also equal to the value of what comes in through the source. Therefore, we can instead for mulate the problem as follows: we add an edge $e^* = (t, s)$ with ∞ capacity. If $f(e^*) = \text{val}(f)$, we have

conservation at s and t as well. Therefore, by maximizeing the value of the flow in the original problem, with this formulation we simply aim to maximize $f(e^*)$, where we impose conservation for all $v \in V$.

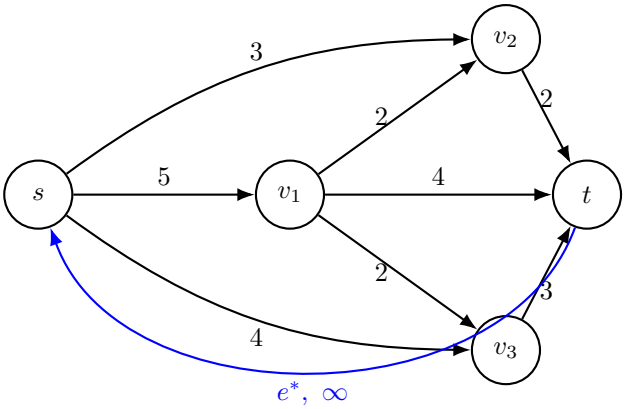


Figure 5.1: An example of formulation

Note that this formulation makes sense since if we can maximize $f(e^*)$ with all vertices satisfy conservation property and all edges satisfy capacity property, then $f(e^*) = \text{val}(f)$ if we consider f only on original graph without e^*.

5.1 Augmenting a flow

Definition 5.1.1. Let f be a feasible flow in $(\vec{D}, s, t, c)$. Let G be the undirected underlying multigraph of D. An s,t-path in G

$$s = v_0 e_1 v_1 e_2 v_2 \dots e_k v_k = t$$

is an *f-augmenting path* if for every forwards edge $\vec{e_i}$ (i.e. that $\vec{e_i} = (v_{i-1}, v_i)$ in $\vec{D}$) we have $f(\vec{e_i}) < c(\vec{e_i})$, and for every backwards edge $\vec{e_i}$ (i.e. that $\vec{e_i} = (v_i, v_{i-1})$ in $\vec{D}$) we have $f(\vec{e_i}) > 0$. The *tolerance* of the path is

$$\min\left\{\{c(\vec{e_i}) - f(\vec{e_i}) : \vec{e_i} \text{ is forwards}\} \cup \{f(\vec{e_i}) : \vec{e_i} \text{ is backwards}\}\right\}.$$

Observation. Any f-augmenting path for some feasible flow f has positive tolerance.

Lemma 5.1.1 (Augmenting lemma). Let f be a feasible flow in $(\vec{D}, s, t, c)$, and let P be an f-augmenting path with tolerance z. Then, if f' is defined by

$$f'(\vec{e}) = \begin{cases} f(\vec{e}) + z, & \text{if } \vec{e} \in P \text{ is forwards;} \\ f(\vec{e}) - z, & \text{if } \vec{e} \in P \text{ is backwards;} \\ f(\vec{e}), & \text{otherwise.} \end{cases}$$

Then, f' is feasible with $\text{val}(f') = \text{val}(f) + z$.

Proof.

(i) f' is feasible:

- $0 \le f'(\vec{e}) \le c(\vec{e})$ is true by our choice of z. ($\vec{e} \notin P$: $f(\vec{e})$ is unchanged and f is feasible, so it is true. $\vec{e} \in P$: z is the minimum of the tolerance along all edges, so $\vec{e}$ can afford to change by z.)

– Conservation: for $v \neq s, t$, consider

$$\sum_{\vec{e}:e^+=v} f'(\vec{e}) - \sum_{\vec{e}:e^-=v} f'(\vec{e}).$$

If $v \notin P$, then everything is the same; if $v \in P$, by discussing each of 4 possible orientations of the edges in P incident to v in $\vec{D}$, we have the equality. More specifically, if $v = v_i$ for some $1 \leq i \leq k-1$, then $v_{i-1}e_iv_ie_{i+1}v_{i+1} \subseteq P$, and if e_i, e_{i+1} are both forwards edges or both backwards edges, then

$$\sum_{\vec{e}:e^+=v} f'(\vec{e}) - \sum_{\vec{e}:e^-=v} f'(\vec{e}) = \left(\sum_{\vec{e}:e^+=v} f(\vec{e}) \pm z\right) - \left(\sum_{\vec{e}:e^-=v} f(\vec{e}) \pm z\right).$$

If one of e_i, e_{i+1} is forwards and the other is backwards, then

$$\sum_{\vec{e}:e^+=v} f'(\vec{e}) - \sum_{\vec{e}:e^-=v} f'(\vec{e}) = \begin{cases} \left(\sum_{\vec{e}:e^+=v} f(\vec{e}) + z - z\right) - \sum_{\vec{e}:e^-=v} f(\vec{e}), & \text{if } e_i \text{ forwards;} \\ \sum_{\vec{e}:e^+=v} f(\vec{e}) - \left(\sum_{\vec{e}:e^-=v} f(\vec{e}) + z - z\right), & \text{if } e_i \text{ backwards.} \end{cases}$$

(ii) Value: Consider

$$\mathrm{val}(f') = \sum_{\vec{e}:e^+=t} f'(\vec{e}) - \sum_{\vec{e}:e^-=t} f'(\vec{e}).$$

Then no matter the edge in P incident to t is forwards or backwards we can see that $\mathrm{val}(f') = \mathrm{val}(f) + z$ by definition of f'.

■

Lemma 5.1.2 (Characteriazation lemma)**.** A feasible flow f in $(\vec{D}, s, t, c)$ is of maximum value iff there is no f-augmenting path (with positive tolerance).

Proof.

($\Rightarrow$) If f is of maximum value and there is some f-augmenting path, then by augmenting lemma we can have a flow f' of larger value, which is a contradiction.

($\Leftarrow$) If there is no f-augmenting path, let

$$S_f = \{v \in V : \exists f\text{-augmenting path from } s \text{ to } v\} \, (s \in S_f, t \notin S_f).$$

Claim 5.1.1. $\mathrm{cap}(S_f, \overline{S_f}) = \mathrm{val}(f)$.

Proof. By the conservation lemma,

$$\mathrm{val}(f) = \sum_{\vec{e} \in [S_f, \overline{S_f}]} f(\vec{e}) - \sum_{\vec{e} \in [\overline{S_f}, S]} f(\vec{e}) = \sum_{\vec{e} \in [S_f, \overline{S_f}]} c(\vec{e}) - \sum_{\vec{e} \in [\overline{S_f}, S_f]} 0 = \mathrm{cap}([S_f, \overline{S_f}]).$$

since for each edge $\vec{e}$ between S_f and $\overline{S_f}$, it cannot be forwards edge with $f(\vec{e}) < c(\vec{e})$ or backwards edge with $f(\vec{e}) > 0$, otherwise there is a f-augmenting path from s to the endpoint of $\vec{e}$ in $\overline{S_f}$, which is a contradiction. ⊛

Now since the claim holds, if f^* is a max-value flow, then by weak duality, we have

$$\mathrm{val}(f^*) \leq \mathrm{cap}(S_f, \overline{S_f}) = \mathrm{val}(f),$$

so f is indeed of maximum value.

■

Then, we have the following corollary:

Theorem 5.1.1 (Ford-Fulkerson, 1956). For any network flow problem $(\vec{D}, s, t, c)$, we have

$$\max_{\text{feasible flow } f} \operatorname{val}(f) = \min_{\text{source-sink cut } [S,\overline{S}]} \operatorname{cap}(S, \overline{S}).$$

Proof.

- $\leq$: weak duality.
- $\geq$: Let f^* be a max-value flow. Let $[S_{f^*}, \overline{S_{f^*}}]$ be the source-sink cut from characterization lemma. Then the claim implies $\operatorname{cap}(S_{f^*}, \overline{S_{f^*}}) = \operatorname{val}(f^*)$.

■

Application: Local edge-Menger theorem In any graph G, for any $u, v \in V(G)$, we have

$$\kappa'(u, v) = \lambda'(u, v).$$

(Here $\kappa'(u, v)$ is the minimum number of edges in a u, v-separator, and $\lambda'(u, v)$ is the maximum number of pairwise edge-disjoint u, v-paths.)

We construct an instance of a network flow problem: $\vec{D} = (v(G), \vec{E})$ where

$$\vec{E} = \bigcup_{e=\{x,y\} \in E(G)} \{(x, y), (y, x)\},$$

and $s = u, t = v$. We let $c \equiv 1$ (capacity of each edge is 1).

Claim 5.1.2. $\lambda'(u, v) \geq \max \operatorname{val}(f)$ and $\kappa'(u, v) = \min \operatorname{cap}(S, \overline{S})$.

Proof. Let $F \subseteq E$ be a minimum u, v-separator. Hence, $|F| = \kappa'(u, v)$. Let S be the component of u in $G - F$; then $v \notin S$. Then, we have $[S, \overline{S}] = F$, and so $\operatorname{cap}(S, \overline{S}) = |F| = \kappa'(u, v)$.

Conversely, for any source-sink cut $[S, \overline{S}]$, $[S, \overline{S}]$ is a u, v-separaotr in G, and thus

$$\kappa'(u, v) \leq \left|[S, \overline{S}]\right| = \operatorname{cap}(S, \overline{S}).$$

On the other hand, we know $\min \operatorname{cap}(S, \overline{S})$ is an integer, and by Ford-Fulkerson, $\max \operatorname{val}(f) = \min \operatorname{cap}(S, \overline{S})$, so $\max \operatorname{val}(f)$ is an integer.

> **Question.** Can we decompose it into edge-disjoint paths from u to v?

We may assume that for every $\{x, y\} \in E$, $f(x, y)f(y, x) = 0$. (This is because WLOG if $f(x, y) = f_1$ and $f(y, x) = f_2$ with $f_1 \geq f_2$, then we may instead set $f(x, y) = f_1 - f_2$ and $f(y, x) = 0$.)

Theorem 5.1.2 (Integrality theorem). If $(\vec{D}, s, t, c)$ is a network with integer capacities, then there is a maximum value flow f^* whose values of all edges are all integral, and moreover we can decompose the flow

$$f^* = g_1 + g_2 + \cdots + g_{\operatorname{val}(f^*)},$$

where each g_i is a unit s, t-flow (all edges are of integer value).

Assuming the integrality theorem holds, the proof follows: in out Menger network, we have integer capacities, so we have a maximum flow with a decomposition into $\operatorname{val}(f^*)$ unit flows. Note that for each g_i, it is a unit u, v-flow, so if we collect all edges e has $g_i(e) = 1$, then it forms a

u, v-path. Also, each path given by g_i for all i are pairwise edge-disjoint because capacities are 1, and thus we cannot reuse edges here. ⊛

This claim shows

$$\kappa'(u, v) = \min \operatorname{cap}(S, \overline{S}) = \max \operatorname{val}(f) \leq \lambda'(u, v),$$

and note that $\lambda'(u, v) \geq \kappa'(u, v)$ is trivial since if we have $\lambda'(u, v)$ pairwise edge-disjoint u, v-path, we at least have to take one edge out from each path to disconnect u and v. Thus, we have proved the local edge-Menger theorem.

Appendix